

D E V O P S S H A C K

DevSecOps

50 Hands-On Assignments

Complete Workbook with Solutions & Ultra-Deep Explanations

Gitleaks · Trivy · Checkov · SonarQube · Vault · OPA
Falco · Cosign · Syft · ArgoCD · Istio · AWS Security

Author: [DevOps Shack](#) | [Aditya Jaiswal](#)

devopsshack.com

Instagram | LinkedIn | Twitter : [@devopsshack](#)

Table of Contents

01. Detect Hardcoded Secrets with Gitleaks [Secrets Management]
02. Scan Docker Images with Trivy [Container Security]
03. Terraform Security Scanning with Checkov [IaC Security]
04. Static Code Analysis with SonarQube [Code Quality & Security]
05. Manage Secrets with HashiCorp Vault [Secrets Management]
06. Dynamic Application Security Testing with OWASP ZAP [DAST]
07. Kubernetes Security Policies with OPA/Gatekeeper [Container Security]
08. Network Policy Enforcement in Kubernetes [Network Security]
09. Secure GitHub Actions Workflows [CI/CD Security]
10. Kubernetes CIS Benchmark with kube-bench [Compliance]
11. Runtime Threat Detection with Falco [Runtime Security]
12. SBOM Generation and Verification [Supply Chain]
13. Kubernetes RBAC Deep Dive [Identity & Access]
14. External Secrets Operator with AWS Secrets Manager [Secrets Management]
15. Terraform State Security and Remote Backends [IaC Security]
16. Signing Container Images with Cosign [Image Security]
17. Security Monitoring with Prometheus and Grafana [Monitoring]
18. CVE Management with Dependency Track [Vulnerability Management]
19. Service Mesh Security with Istio mTLS [Network Security]
20. Cloud Security Posture Management with AWS SecurityHub [Compliance]
21. Pod Security Standards and Admission Control [Container Security]
22. Jenkins Pipeline Security Hardening [CI/CD Security]
23. TLS Certificate Management with cert-manager [Cryptography]
24. Centralized Security Logging with ELK Stack [Log Management]
25. AWS IAM Least Privilege with AWS Access Analyzer [Access Control]
26. Secure GitOps with ArgoCD and Vault [GitOps Security]
27. API Security Testing with Nuclei [DAST]
28. Container Hardening with Distless and Rootless Images [Container Security]
29. Kubernetes Manifest Security Scanning with Kubesec [IaC Security]
30. Security Incident Response Runbook [Incident Response]
31. AWS GuardDuty Threat Detection [Cloud Security]
32. SOC 2 Controls Implementation for DevOps [Compliance]
33. Multi-stage Dockerfile Security Best Practices [Container Security]
34. AWS Secrets Manager Auto-Rotation [Secrets Management]
35. Web Application Firewall with AWS WAF [Network Security]
36. Dependency Vulnerability Scanning with Snyk [CI/CD Security]
37. Kubernetes Audit Logging Deep Dive [Kubernetes Security]
38. OIDC Federated Identity for GitHub Actions to AWS [Identity & Access]
39. DORA Metrics for DevSecOps Teams [Compliance]
40. License Compliance Scanning with FOSSA [Scanning]
41. Kubernetes Admission Controllers for Security [Runtime Security]

- 42. Data Encryption at Rest and in Transit [Encryption]
 - 43. S3 Bucket Security Audit and Remediation [Cloud Security]
 - 44. Secure Docker Build with BuildKit and SBOM [CI/CD Security]
 - 45. Kubernetes Penetration Testing with kube-hunter [Penetration Testing]
 - 46. SIEM Integration with Splunk for DevSecOps [Monitoring]
 - 47. Open Policy Agent for Microservice Authorization [Policy as Code]
 - 48. End-to-End DevSecOps Pipeline Architecture [Vulnerability Management]
 - 49. AWS EKS Security Hardening [Cloud Security]
 - 50. Complete DevSecOps Pipeline: End-to-End Project [Capstone]
-

Introduction to DevSecOps

DevSecOps is the philosophy and practice of integrating security throughout the entire software development lifecycle (SDLC), rather than treating it as a final gate before deployment. The term — Development, Security, Operations — reflects the cultural and technical merger of three disciplines that historically operated in isolation.

The Cost of Security Debt

The National Institute of Standards and Technology (NIST) data consistently shows that fixing a bug found in production costs 100x more than fixing it during design, and 15x more than fixing it during coding. Security vulnerabilities follow the same curve: a misconfigured S3 bucket caught by Checkov during IaC review costs 30 minutes to fix. The same misconfiguration discovered after a data breach costs millions in breach response, regulatory fines, customer notification, and reputational damage.

The DevSecOps Toolchain

This workbook covers the complete DevSecOps toolchain organized by pipeline stage:

Pipeline Stage	Security Tools Covered
Pre-commit	Gitleaks, pre-commit hooks, Husky
Source Control	GitHub Advanced Security, CodeQL, Dependabot
Build	Trivy (image scan), Checkov (IaC), SonarQube (SAST), Snyk (SCA)
Test	OWASP ZAP (DAST), Nuclei (API testing), kube-hunter
Artifacts	Cosign (signing), Syft (SBOM), Trivy (image scan)
Deploy	OPA Gatekeeper, Kyverno, ArgoCD with policy gates
Runtime	Falco (threat detection), Istio mTLS, NetworkPolicy
Monitoring	Prometheus, Grafana, Splunk SIEM, AWS GuardDuty
Compliance	CIS Benchmarks, kube-bench, AWS SecurityHub, Vault audit

How to Use This Workbook

Each assignment follows a consistent structure: Objective (what you will achieve), Background & Theory (the 'why' behind the tool and approach), Tasks (step-by-step instructions), Complete Solution (working code and configurations), Ultra-Deep Explanation (internals, architecture, and security principles), Best Practices, and Common Mistakes.

Assignments are designed for hands-on execution. You will need: a workstation with Docker, kubectl, Terraform, Git, and Python installed; a GitHub account; and optionally an AWS account (free tier sufficient for most assignments). Each assignment builds on previous ones, so completing them in order is recommended.

ASSIGNMENT 1 · SECRETS MANAGEMENT

Detect Hardcoded Secrets with Gitleaks

Difficulty: ★ Beginner · Tools: Gitleaks, Git, GitHub Actions

Objective

Install Gitleaks and scan a Git repository to detect hardcoded secrets such as API keys, passwords, and tokens before they reach remote branches.

Background & Theory

Secrets sprawl is one of the most prevalent security failures in modern software delivery. Developers routinely hardcode database passwords, cloud credentials, and API tokens directly in source code. Once committed, even deleted secrets live forever in Git history.

Gitleaks is an open-source SAST tool that scans Git repositories using a library of regex-based rules and entropy analysis to identify exposed secrets across all commits, branches, and tags.

The shift-left philosophy demands that secrets scanning happens at pre-commit or CI pipeline stage — before code reaches shared repositories, and certainly before it deploys to production environments.

NIST SP 800-53 and CIS Benchmarks both highlight secrets management as a critical control. A single exposed AWS Access Key can result in thousands of dollars in unexpected charges and a full account compromise within minutes of exposure.

Tasks

Task 1: Install and configure Gitleaks

- Install Gitleaks v8.x on your local machine
- Verify installation with `gitleaks version`
- Create a sample repository with intentional secrets

Task 2: Perform local scan

- Run a full repository scan
- Understand the output format (findings JSON)
- Baseline known false positives with `.gitleaks.toml`

Task 3: Integrate into GitHub Actions

- Create `.github/workflows/secret-scan.yml`
- Configure Gitleaks action on `push` and `pull_request` events
- Set the pipeline to fail on detected secrets

✓ Complete Solution

Step 1: Install Gitleaks

bash — Install Gitleaks (Linux/macOS)

```
# Linux (AMD64)
curl -sSL
https://github.com/gitleaks/gitleaks/releases/download/v8.18.4/gitleaks_8.18.4_linux
_x64.tar.gz \
| tar -xz && sudo mv gitleaks /usr/local/bin/

# macOS via Homebrew
brew install gitleaks

# Verify
gitleaks version    # → gitleaks version 8.18.4
```

Step 2: Create Test Repository with Secrets

python — config.py (intentional secrets for testing)

```
# DO NOT DO THIS IN REAL CODE — For testing only
AWS_ACCESS_KEY_ID = "AKIAIOSFODNN7EXAMPLE"
AWS_SECRET_ACCESS_KEY = "wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY"
DB_PASSWORD = "superSecretPassword123!"
GITHUB_TOKEN = "ghp_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"
STRIPE_API_KEY = "sk_live_XXXXXXXXXXXXXXXXXXXXXXXXXXXX"
```

Step 3: Run Gitleaks Scan

bash — Scanning commands

```
# Scan entire repo history
gitleaks detect --source . --verbose

# Scan staged changes only (pre-commit hook)
gitleaks protect --staged --verbose

# Output findings to JSON for CI parsing
gitleaks detect --source . --report-format json \
  --report-path gitleaks-report.json --exit-code 1

# Scan a specific branch
gitleaks detect --source . --log-opts="main..HEAD"
```

Step 4: Custom .gitleaks.toml Configuration

toml — .gitleaks.toml

```
[extend]
```

```
useDefault = true

[[rules]]
id = "devopsshack-custom-api-key"
description = "DevOpsShack Internal API Key"
regex = "'DSK_[a-zA-Z0-9]{32}'"
tags = ["api-key", "internal"]

[allowlist]
description = "Known safe test values"
regexes = [
  "'AKIAIOSFODNN7EXAMPLE'", # AWS docs example key
  "'EXAMPLEKEY'"
]
paths = [
  "'tests/fixtures/*.'",
  "'docs/examples/*.'"
]
```

Step 5: GitHub Actions Integration

yaml — .github/workflows/secret-scan.yml

```
name: Secret Scanning with Gitleaks

on:
  push:
    branches: [ "*" ]
  pull_request:
    branches: [ main, develop ]

jobs:
  gitleaks-scan:
    name: Detect Secrets
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout code
        uses: actions/checkout@v4
        with:
          fetch-depth: 0 # Full history for complete scan

      - name: Run Gitleaks
        uses: gitleaks/gitleaks-action@v2
        env:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
          GITLEAKS_LICENSE: ${ secrets.GITLEAKS_LICENSE } # For org scan

      - name: Upload SARIF results
        if: always()
        uses: github/codeql-action/upload-sarif@v3
```



```
with:
  sarif_file: results.sarif
```

Step 6: Pre-commit Hook Setup

bash — Install pre-commit hook

```
# Install pre-commit framework
pip install pre-commit

# .pre-commit-config.yaml
cat > .pre-commit-config.yaml << 'EOF'
repos:
  - repo: https://github.com/gitleaks/gitleaks
    rev: v8.18.4
    hooks:
      - id: gitleaks
EOF

# Activate hooks
pre-commit install
pre-commit install --hook-type pre-push
```

Ultra-Deep Explanation

How Gitleaks Detection Engine Works

Gitleaks uses two complementary detection mechanisms: regex pattern matching and Shannon entropy analysis. The regex library ships with 100+ built-in rules covering AWS keys, GCP service accounts, GitHub tokens, Stripe keys, Slack webhooks, and more. Each rule has a confidence score.

Shannon entropy measures the randomness of a string. High-entropy strings (>3.5 bits/char) that match a suspicious keyword context (password=, secret=, key=) are flagged even without a regex match. This catches obfuscated or custom secret formats.

Git History Scanning Deep Dive

When Gitleaks runs with `--source` on a Git repo, it iterates through every commit object via git log. For each commit diff, it checks added lines against all rules. This means secrets deleted in commit N but added in commit N-3 are still detected — a critical capability since git history is permanent.

The `--fetch-depth: 0` in GitHub Actions is mandatory. Shallow clones (the default) only retrieve recent commits and will miss historical secrets. Always use full history for security scanning.

SARIF Integration with GitHub Security

SARIF (Static Analysis Results Interchange Format) is a JSON standard for security tool output. When you upload SARIF to GitHub, findings appear in the Security → Code Scanning Alerts tab with severity levels, affected files, and suggested fixes. This enables security team triage without reading raw JSON.

False Positive Management Strategy

In practice, any secrets scanner generates false positives on test fixtures, documentation examples, and generated files. The allowlist in `.gitleaks.toml` accepts regexes (for specific values) and paths (glob patterns). Always prefer path-based allowlisting over value-based — it is easier to audit and does not accidentally whitelist real secrets that share a pattern with test values.

★ Best Practices

- Always use `--fetch-depth: 0` in CI to scan full Git history
- Store `.gitleaks.toml` in the repository root for consistency across environments
- Rotate any secret found by Gitleaks immediately — detection does not undo exposure
- Combine Gitleaks with HashiCorp Vault or AWS Secrets Manager for proper secret lifecycle
- Run Gitleaks as a pre-commit hook AND in CI for defense-in-depth
- Review SARIF alerts weekly and track mean-time-to-remediation (MTTR) as a KPI

⚠ Common Mistakes to Avoid

- Shallow clone (`fetch-depth: 1`) in GitHub Actions misses historical secrets — always use `fetch-depth: 0`
- Blocking only on high-severity findings — low severity secrets (e.g., internal tokens) are equally dangerous
- Never adding `.gitleaks.toml` — default rules generate significant noise in monorepos
- Forgetting to rotate the secret after detection — Gitleaks finding is an alarm, not a fix
- Using the same Gitleaks config locally and in CI without testing allowlist regexes properly

ASSIGNMENT 2 · CONTAINER SECURITY

Scan Docker Images with Trivy

Difficulty: ★ Beginner · Tools: Trivy, Docker, GitHub Actions, SARIF

Objective

Use Trivy to perform comprehensive vulnerability scanning of Docker images, including OS packages, language dependencies, misconfigurations, and secrets embedded in container layers.

Background & Theory

Container images aggregate vulnerabilities from multiple sources: the base OS layer (Ubuntu, Alpine, RHEL UBI), language runtime packages (npm, pip, Maven), application dependencies, and even secrets accidentally baked into layers. A single unpatched base image can expose hundreds of CVEs.

Trivy (by Aqua Security) is the industry's most widely adopted open-source vulnerability scanner. It supports Docker images, filesystems, Git repositories, Kubernetes manifests, Terraform, CloudFormation, and more — making it a Swiss Army knife for DevSecOps teams.

CVE (Common Vulnerabilities and Exposures) is a dictionary of publicly known cybersecurity vulnerabilities. Each CVE has a CVSS score (0-10), exploitability metrics, and remediation guidance. Trivy maps findings to CVE identifiers from multiple databases: NVD, GitHub Advisory, RedHat, Alpine, and more.

Supply chain attacks (SolarWinds, Log4Shell, XZ Utils) have elevated container scanning from a nice-to-have to a regulatory requirement in frameworks like PCI-DSS v4.0, SOC 2 Type II, and FedRAMP.

Tasks

Task 1: Install and run Trivy locally

- Install Trivy on your workstation
- Pull a test Docker image (nginx:1.24)
- Run an image vulnerability scan and analyze the output

Task 2: Advanced Trivy scanning modes

- Scan for misconfigurations in Dockerfile
- Scan secrets embedded in image layers
- Generate SBOM (Software Bill of Materials) in CycloneDX format

Task 3: GitHub Actions pipeline integration

- Create trivy-scan.yml workflow
- Configure severity thresholds to fail the pipeline

- Upload results to GitHub Security dashboard via SARIF

✓ Complete Solution

Step 1: Install Trivy

bash — Trivy installation

```
# Linux (Debian/Ubuntu)
sudo apt-get install wget apt-transport-https gnupg lsb-release
wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key | sudo apt-key
add -
echo "deb https://aquasecurity.github.io/trivy-repo/deb $(lsb_release -sc) main" \
| sudo tee -a /etc/apt/sources.list.d/trivy.list
sudo apt-get update && sudo apt-get install trivy

# macOS
brew install trivy

# Docker (no install required)
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
  aquasec/trivy:latest image nginx:1.24

# Verify
trivy --version    # → Version: 0.50.x
```

Step 2: Image Vulnerability Scan

bash — Scanning Docker images

```
# Basic scan
trivy image nginx:1.24

# Filter by severity (fail on CRITICAL/HIGH)
trivy image --severity CRITICAL,HIGH nginx:1.24

# Output formats: table, json, sarif, template, cyclonedx, spdx
trivy image --format json --output trivy-report.json nginx:1.24

# Ignore unfixed vulnerabilities (not patched upstream yet)
trivy image --ignore-unfixed nginx:1.24

# Scan a locally built image
docker build -t myapp:latest .
trivy image --severity CRITICAL,HIGH,MEDIUM myapp:latest

# Scan image tarball (useful in air-gapped environments)
docker save myapp:latest -o myapp.tar
trivy image --input myapp.tar
```

Step 3: Dockerfile Misconfiguration Scan

dockerfile — Dockerfile (with intentional issues)

```
FROM ubuntu:20.04

# Issue 1: Running as root
# Issue 2: Using ADD instead of COPY
# Issue 3: No USER instruction

RUN apt-get update && apt-get install -y curl wget

ADD https://example.com/app.tar.gz /app/

EXPOSE 22 # Issue: exposing SSH

CMD ["/app"]
```

bash — Misconfiguration and secret scanning

```
# Scan Dockerfile for misconfigurations
trivy config --severity HIGH,CRITICAL .

# Scan image layers for secrets
trivy image --scanners secret myapp:latest

# Combined scan: vulns + misconfig + secrets
trivy image --scanners vuln,secret,config \
  --severity CRITICAL,HIGH myapp:latest

# Generate CycloneDX SBOM
trivy image --format cyclonedx \
  --output sbom.json myapp:latest
```

Step 4: GitHub Actions Full Integration

yaml — .github/workflows/trivy-scan.yml

```
name: Container Security Scan

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 6 * * 1' # Weekly scan on Mondays

jobs:
  trivy-scan:
    name: Trivy Vulnerability Scan
    runs-on: ubuntu-latest
```

```
permissions:
  contents: read
  security-events: write
  actions: read

steps:
  - name: Checkout code
    uses: actions/checkout@v4

  - name: Build Docker image
    run: docker build -t ${{ github.repository }}:${{ github.sha }} .

  - name: Run Trivy vulnerability scan (SARIF)
    uses: aquasecurity/trivy-action@master
    with:
      image-ref: '${{ github.repository }}:${{ github.sha }}'
      format: 'sarif'
      output: 'trivy-results.sarif'
      severity: 'CRITICAL,HIGH'
      ignore-unfixed: true

  - name: Upload SARIF to GitHub Security
    uses: github/codeql-action/upload-sarif@v3
    if: always()
    with:
      sarif_file: 'trivy-results.sarif'

  - name: Run Trivy scan (exit code for pipeline gate)
    uses: aquasecurity/trivy-action@master
    with:
      image-ref: '${{ github.repository }}:${{ github.sha }}'
      format: 'table'
      exit-code: '1'
      severity: 'CRITICAL'
      ignore-unfixed: true

  - name: Upload SBOM artifact
    uses: actions/upload-artifact@v4
    with:
      name: sbom-cyclonedx
      path: sbom.json
```

Step 5: Trivy .trivyignore Configuration

text — .trivyignore (suppress known false positives)

```
# Suppress specific CVEs with justification
CVE-2023-44487 # HTTP/2 Rapid Reset — mitigated at load balancer level
CVE-2021-44228 # Log4Shell — not using Log4j in this service

# Suppress by package name
# pkg:deb/debian/libssl1.1@1.1.1n-0+deb10u6
```

Ultra-Deep Explanation

Trivy's Multi-Database Architecture

Trivy aggregates vulnerability data from NVD (National Vulnerability Database), GitHub Security Advisories, OS-specific advisories (Alpine SecDB, RedHat OVAL, Debian Security Tracker, Ubuntu USN), and language ecosystem registries (npm audit, PyPI Safety DB, Go Vuln DB). This multi-source approach gives higher coverage than single-database scanners.

How Image Layer Scanning Works

A Docker image is a stack of read-only layers (each FROM, RUN, COPY instruction creates a layer). Trivy unpacks every layer using the image manifest and scans each layer independently. This means a secret or package installed in an intermediate layer and deleted in a later layer is still detected — because deleted files are not removed from the image, only hidden from the running container filesystem.

CVSS Score Interpretation

CVSS v3.1 scores: 9.0-10.0 = Critical, 7.0-8.9 = High, 4.0-6.9 = Medium, 0.1-3.9 = Low. However, raw CVSS score is not sufficient for prioritization. Trivy also reports exploitability (is there a working exploit available?), fixability (is there a patched version?), and reachability (is the vulnerable code path actually executed?). Prioritize: Critical + exploitable + fixable first.

SBOM and Supply Chain Compliance

A Software Bill of Materials (SBOM) is a machine-readable inventory of all components in a software artifact. US Executive Order 14028 and NTIA guidelines mandate SBOMs for software sold to federal agencies. CycloneDX and SPDX are the two dominant SBOM standards. Trivy can generate both, enabling automated license compliance checks and dependency audits.

★ Best Practices

- Use minimal base images (distroless, Alpine, scratch) to reduce attack surface
- Pin base image versions with SHA256 digest: FROM ubuntu@sha256:abc123...
- Separate Trivy runs: one for SARIF (always runs), one with exit-code for pipeline gate
- Schedule weekly baseline scans to catch newly disclosed CVEs in deployed images
- Integrate SBOM generation into every build for supply chain traceability
- Use .trivyignore to suppress false positives with justification comments

⚠ Common Mistakes to Avoid

- Using latest tag for base images — makes vulnerability baselines unpredictable
- Not running --ignore-unfixed — clutters reports with CVEs that have no patch available
- Single severity threshold across all services — critical APIs need CRITICAL gate, internal tools can use HIGH
- Not updating Trivy's vulnerability database before scanning (trivy image --download-db-only)
- Treating Trivy findings as the only security signal — complement with runtime monitoring (Falco)

ASSIGNMENT 3 · IAC SECURITY**Terraform Security Scanning with Checkov**

Difficulty: ★★ Intermediate · Tools: Checkov, Terraform, GitHub Actions, tfsec

Objective

Implement infrastructure-as-code security scanning using Checkov to detect misconfigurations in Terraform code before cloud resources are provisioned.

Background & Theory

Infrastructure as Code (IaC) security misconfigurations are responsible for a significant portion of cloud breaches. Public S3 buckets, open security groups, unencrypted databases, and disabled audit logging are common Terraform mistakes that Checkov detects before terraform apply.

Checkov (by Bridgecrew/Palo Alto Networks) is a static code analysis tool for IaC. It supports Terraform, CloudFormation, ARM, Kubernetes, Dockerfile, and Helm charts. It ships with 1000+ built-in policies mapped to CIS Benchmarks, NIST, SOC 2, PCI-DSS, HIPAA, and more.

The CIS AWS Foundations Benchmark is the gold standard for AWS security. It covers 60+ controls across IAM, logging, networking, and storage. Checkov maps each policy to a CIS control ID (e.g., CKV_AWS_18 = S3 bucket access logging enabled).

Shift-left for IaC means catching a public S3 bucket in a pull request review, not after it exposes customer data to the internet. The cost of fixing a misconfiguration in code is near-zero; the cost after a breach can be millions.

Tasks

Task 1: Write intentionally misconfigured Terraform

- Create an S3 bucket without encryption or access logging
- Create a security group with 0.0.0.0/0 ingress on all ports
- Create an RDS instance without encryption or backup enabled

Task 2: Run Checkov and interpret results

- Install Checkov and scan the Terraform directory
- Understand PASSED/FAILED/SKIPPED checks
- Fix the identified misconfigurations

Task 3: CI/CD integration with custom policies

- Add Checkov to GitHub Actions on pull requests
- Write a custom Checkov policy in Python
- Configure .checkov.yml baseline to suppress known issues

✓ Complete Solution

Step 1: Misconfigured Terraform Files

hcl — main.tf (intentionally vulnerable)

```
# Insecure S3 bucket
resource "aws_s3_bucket" "data" {
  bucket = "my-company-data-bucket"
  # ✗ Missing: versioning, encryption, access logging, public access block
}

# Overly permissive security group
resource "aws_security_group" "web" {
  name = "web-sg"
  ingress {
    from_port = 0
    to_port   = 65535
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"] # ✗ Open to the world
  }
}

# Insecure RDS
resource "aws_db_instance" "main" {
  identifier      = "prod-db"
  engine          = "mysql"
  instance_class  = "db.t3.micro"
  allocated_storage = 20
  username        = "admin"
  password        = "password123" # ✗ Hardcoded password
  publicly_accessible = true      # ✗ Exposed to internet
  skip_final_snapshot = true      # ✗ No backup on delete
  # ✗ Missing: storage_encrypted, backup_retention_period, deletion_protection
}

# Insecure IAM role
resource "aws_iam_policy" "broad" {
  name = "broad-policy"
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Action = "*" # ✗ Wildcard actions
      Resource = "*" # ✗ Wildcard resources
    }]
  })
}
```

Step 2: Install Checkov and Scan

bash — Install and run Checkov

```
# Install Checkov
pip install checkov

# Basic directory scan
checkov -d ./terraform

# Scan with specific framework
checkov -d . --framework terraform

# Output as JSON for CI
checkov -d . --output json > checkov-report.json

# Filter by severity
checkov -d . --check CKV_AWS_18,CKV_AWS_19 # Specific checks

# Scan single file
checkov -f main.tf

# Fail only on HIGH severity (using --check-severity)
checkov -d . --check-severity HIGH

# Generate SARIF output
checkov -d . --output sarif > checkov.sarif
```

Step 3: Fixed Terraform Configuration

hcl — main.tf (secure version)

```
# Secure S3 bucket
resource "aws_s3_bucket" "data" {
  bucket = "my-company-data-bucket"
  tags = { Environment = "production" }
}

resource "aws_s3_bucket_versioning" "data" {
  bucket = aws_s3_bucket.data.id
  versioning_configuration { status = "Enabled" }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "data" {
  bucket = aws_s3_bucket.data.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "aws:kms"
    }
  }
}

resource "aws_s3_bucket_public_access_block" "data" {
  bucket = aws_s3_bucket.data.id
  block_public_acls = true
  block_public_policy = true
  ignore_public_acls = true
  restrict_public_buckets = true
}
```

```
}

resource "aws_s3_bucket_logging" "data" {
  bucket      = aws_s3_bucket.data.id
  target_bucket = aws_s3_bucket.logs.id
  target_prefix = "s3-access-logs/"
}

# Restricted security group
resource "aws_security_group" "web" {
  name = "web-sg"
  ingress {
    from_port = 443
    to_port   = 443
    protocol  = "tcp"
    cidr_blocks = ["10.0.0.0/8"] # ☒ Internal CIDR only
  }
  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

# Secure RDS
resource "aws_db_instance" "main" {
  identifier      = "prod-db"
  engine          = "mysql"
  engine_version  = "8.0.35"
  instance_class  = "db.t3.micro"
  allocated_storage = 20
  username        = "admin"
  password        = random_password.db.result # ☒ Generated
  publicly_accessible = false # ☒ Private
  storage_encrypted = true # ☒ Encrypted
  backup_retention_period = 7 # ☒ 7-day backup
  deletion_protection = true # ☒ Protected
  skip_final_snapshot = false
  multi_az            = true # ☒ HA
}

resource "random_password" "db" {
  length = 16
  special = true
}
```

Step 4: Custom Checkov Policy

python — custom_checks/check_s3_tags.py

```
from checkov.common.models.enums import CheckResult, CheckCategories
from checkov.terraform.checks.resource.base_resource_check import BaseResourceCheck
```

```
class CheckS3RequiredTags(BaseResourceCheck):
    def __init__(self):
        name = "Ensure S3 buckets have required DevOpsShack tags"
        id = "CKV_DEVOPSSHACK_1"
        supported_resources = ['aws_s3_bucket']
        categories = [CheckCategories.GENERAL_SECURITY]
        super().__init__(name=name, id=id,
                         categories=categories,
                         supported_resources=supported_resources)

    def scan_resource_conf(self, conf):
        tags = conf.get("tags", [{}])
        if isinstance(tags, list):
            tags = tags[0] if tags else {}
            required = {"Environment", "Owner", "CostCenter"}
            missing = required - set(tags.keys())
            if missing:
                return CheckResult.FAILED
            return CheckResult.PASSED

scanner = CheckS3RequiredTags()
```

Step 5: GitHub Actions Integration

yml — .github/workflows/checkov.yml

```
name: IaC Security Scan

on:
  pull_request:
    paths: [ 'terraform/**', '*.tf' ]
  push:
    branches: [ main ]

jobs:
  checkov:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - uses: actions/checkout@v4

      - name: Run Checkov IaC Scan
        uses: bridgecrewio/checkov-action@master
        with:
          directory: terraform/
          framework: terraform
          output_format: sarif
          output_file_path: checkov.sarif
          soft_fail: false
          check: CKV_AWS_18,CKV_AWS_19,CKV_AWS_21
```

```
# skip_check: CKV_AWS_144 # Skip if known false positive

- name: Upload SARIF results
  uses: github/codeql-action/upload-sarif@v3
  if: always()
  with:
    sarif_file: checkov.sarif
```

Ultra-Deep Explanation

Checkov's Policy Engine Architecture

Checkov parses Terraform HCL into an abstract syntax tree (AST) and then walks the resource graph. For each resource type, it runs all applicable checks. Checks are Python classes that implement a `scan_resource_conf` method receiving the resource configuration as a dictionary. This design makes custom policy writing very accessible.

CIS Benchmark Mapping

The CIS AWS Foundations Benchmark v1.5 defines 60+ prescriptive security controls. Checkov policies are mapped to these controls by ID. For example, CKV_AWS_18 (S3 access logging) maps to CIS 2.6. Running Checkov with `--compliance cis_aws_v1.5` produces a compliance report showing percentage adherence to each benchmark section.

Understanding Soft vs Hard Fails

`soft_fail: true` makes Checkov exit with code 0 even on policy violations — useful for initial rollout where you want visibility without blocking deployments. Once your team has addressed the existing debt, switch to `soft_fail: false` to enforce the policy gate. Many teams use a phased approach: soft-fail for 30 days, then hard-fail.

Baseline Files for Managing Technical Debt

In legacy codebases with hundreds of existing violations, a `--baseline checkov.baseline` file records all current violations. Future scans only fail on new violations introduced since the baseline was created. This enables teams to stop the bleeding immediately while working down the backlog, rather than being overwhelmed and disabling the scanner entirely.

★ Best Practices

- Run Checkov in pre-commit hooks AND in pull request CI — catch issues before code review
- Map Checkov findings to compliance frameworks (CIS, PCI, SOC2) for audit reporting
- Use `soft_fail: true` for 30 days during initial rollout to build baseline, then switch to hard fail
- Write custom policies for organization-specific standards (required tags, naming conventions, approved regions)
- Integrate with Terraform Cloud run tasks for policy-as-code enforcement
- Track Checkov pass rate as a DevSecOps maturity metric (target: 95%+ pass rate)

Common Mistakes to Avoid

Skipping all checks with `--skip-check` instead of investigating and fixing root cause
Not pinning Checkov version in CI — policy updates can suddenly break builds
Using hardcoded passwords in Terraform variables — use `aws_secretsmanager_secret_version` or `random_password`
Scanning only `.tf` files and missing `.tfvars` files that may contain sensitive values
Not reviewing custom policy coverage — default checks may miss organization-specific risks

ASSIGNMENT 4 · CODE QUALITY & SECURITY

Static Code Analysis with SonarQube

Difficulty: ★★ Intermediate · Tools: SonarQube, SonarScanner, GitHub Actions, Docker

Objective

Deploy SonarQube using Docker, configure a project, run static analysis to detect security vulnerabilities, code smells, and coverage gaps, then integrate with GitHub Actions for automated PR gate checks.

Background & Theory

SonarQube is an enterprise-grade static application security testing (SAST) platform that analyzes source code in 30+ programming languages for security vulnerabilities, bugs, code smells, and technical debt. It implements the OWASP Top 10 and CWE Top 25 security risk categories.

Quality Gates are SonarQube's enforcement mechanism — a set of conditions that must be met before code can be merged. A standard gate might require: Code Coverage > 80%, no new Critical vulnerabilities, Security Hotspot Review Rate = 100%. Failed gates block PR merges when integrated with GitHub branch protection rules.

The SonarQube security model distinguishes Vulnerabilities (confirmed security issues), Security Hotspots (code requiring manual review), and Bugs (code likely to cause runtime errors). This triaging approach reduces alert fatigue compared to tools that flag everything as critical.

Technical debt is measured in time-to-fix. SonarQube calculates the estimated remediation effort for each issue and aggregates it into a Debt Ratio (total debt / total development time). Teams typically target < 5% debt ratio for healthy codebases.

Tasks

Task 1: Deploy SonarQube with Docker Compose

- Write docker-compose.yml for SonarQube and PostgreSQL
- Configure persistence volumes and environment variables
- Access SonarQube UI and create a project with authentication token

Task 2: Run SonarScanner analysis

- Configure sonar-project.properties
- Run SonarScanner for a Java/Node.js/Python project
- Analyze the dashboard for vulnerabilities and code smells

Task 3: GitHub Actions with Quality Gate enforcement

- Configure SonarQube GitHub integration

- Add sonar analysis step to CI workflow
- Block PR merge on Quality Gate failure

✓ Complete Solution

Step 1: SonarQube Docker Compose Deployment

yml — docker-compose.yml

```
version: '3.8'

services:
  sonarqube:
    image: sonarqube:10.3-community
    container_name: sonarqube
    depends_on:
      - sonardb
    environment:
      SONAR_JDBC_URL: jdbc:postgresql://sonardb:5432/sonarqube
      SONAR_JDBC_USERNAME: sonar
      SONAR_JDBC_PASSWORD: sonar_password_strong_123
      SONAR_ES_BOOTSTRAP_CHECKS_DISABLE: "true"
    volumes:
      - sonarqube_data:/opt/sonarqube/data
      - sonarqube_extensions:/opt/sonarqube/extensions
      - sonarqube_logs:/opt/sonarqube/logs
    ports:
      - "9000:9000"
    ulimits:
      nofile:
        soft: 65536
        hard: 65536
    restart: unless-stopped

  sonardb:
    image: postgres:15-alpine
    container_name: sonardb
    environment:
      POSTGRES_USER: sonar
      POSTGRES_PASSWORD: sonar_password_strong_123
      POSTGRES_DB: sonarqube
    volumes:
      - sonardb_data:/var/lib/postgresql/data
    restart: unless-stopped

volumes:
  sonarqube_data:
  sonarqube_extensions:
  sonarqube_logs:
  sonardb_data:
```

bash — Start SonarQube and initial setup

```
# Required: increase vm.max_map_count for Elasticsearch
sudo sysctl -w vm.max_map_count=524288
sudo sysctl -w fs.file-max=131072

# Persist across reboots
echo "vm.max_map_count=524288" | sudo tee -a /etc/sysctl.conf

# Start services
docker-compose up -d

# Watch logs until startup complete
docker-compose logs -f sonarqube

# Default credentials: admin/admin → change on first login
# Access: http://localhost:9000
```

Step 2: Create Project and Generate Token

bash — SonarQube API token creation

```
# Generate authentication token via API
curl -u admin:new_admin_password \
  -X POST "http://localhost:9000/api/user_tokens/generate" \
  -d "name=github-actions-token&type=GLOBAL_ANALYSIS_TOKEN"

# Response: { "token": "sqp_XXXXXXXXXXXXXXXXXXXXXXXXXXXX" }

# Store token as GitHub secret: SONAR_TOKEN
# Store SonarQube URL as GitHub secret: SONAR_HOST_URL
```

Step 3: Configure sonar-project.properties

properties — sonar-project.properties

```
# Project identification
sonar.projectKey=devopsshack-nodejs-app
sonar.projectName=DevOpsShack Node.js Application
sonar.projectVersion=1.0.0

# Source and test paths
sonar.sources=src
sonar.tests=tests
sonar.exclusions=**/node_modules/**,**/dist/**,**/*.test.js,**/coverage/**

# Language settings
sonar.language=js
sonar.javascript.lcov.reportPaths=coverage/lcov.info

# Coverage configuration
sonar.coverage.exclusions=**/*.test.js,**/migrations/**

# Security configuration
```

```
sonar.security.hotspots.inheritFromParent=true
```

Step 4: Run SonarScanner

bash — Run analysis locally

```
# Install SonarScanner
curl -sSL https://binaries.sonarsource.com/Distribution/sonar-scanner-cli/\
sonar-scanner-cli-6.0.0.4432-linux.zip -o sonar-scanner.zip
unzip sonar-scanner.zip && sudo mv sonar-scanner-6.0.0.4432-linux /opt/sonar-scanner
export PATH=$PATH:/opt/sonar-scanner/bin

# Run scanner (reads sonar-project.properties automatically)
sonar-scanner \
  -Dsonar.host.url=http://localhost:9000 \
  -Dsonar.token=sqp_XXXXXXXXXXXXX

# For Maven projects
mvn sonar:sonar \
  -Dsonar.host.url=http://localhost:9000 \
  -Dsonar.token=$SONAR_TOKEN

# For Python with coverage
pytest --cov=src --cov-report=xml:coverage.xml
sonar-scanner -Dsonar.python.coverage.reportPaths=coverage.xml
```

Step 5: GitHub Actions with Quality Gate

yaml — .github/workflows/sonarqube.yml

```
name: SonarQube Code Analysis

on:
  push:
    branches: [ main, develop ]
  pull_request:
    types: [ opened, synchronize, reopened ]

jobs:
  sonarqube:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Install dependencies and run tests
        run: |
```

```
npm ci
npm run test -- --coverage --coverageReporters=lcov

- name: SonarQube Scan
  uses: SonarSource/sonarqube-scan-action@master
  env:
    SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
    SONAR_HOST_URL: ${ secrets.SONAR_HOST_URL }

- name: SonarQube Quality Gate Check
  uses: SonarSource/sonarqube-quality-gate-action@master
  timeout-minutes: 5
  env:
    SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
    SONAR_HOST_URL: ${ secrets.SONAR_HOST_URL }
```

Ultra-Deep Explanation

SonarQube's Analysis Engine

SonarQube uses language-specific parsers to build a semantic model of the code. Unlike regex-based tools, it understands data flow — it can trace a variable from an HTTP request parameter all the way to a database query to identify SQL injection vulnerabilities. This taint analysis (tracking untrusted data through the code) is what distinguishes SAST from simple pattern matching.

Quality Gate Deep Dive

The Quality Gate evaluates conditions on NEW code only (code changed in the current analysis). This is the key insight — it prevents the death spiral where a team inherits 10,000 issues and disables all scanning because they cannot get to green. The New Code period is configurable (previous version, specific date, or 30 days). Teams can achieve compliance on new code while paying down legacy debt separately.

Security Hotspots vs Vulnerabilities

Vulnerabilities are confirmed security issues requiring immediate fixing (e.g., SQL injection with user input directly concatenated into a query). Security Hotspots are areas that require human review — the code uses a security-sensitive API (cryptography, authentication, file access) but SonarQube cannot determine if it is used correctly without context. All hotspots must be reviewed and marked Safe or changed, achieving 100% review rate.

PostgreSQL vs H2 Database

SonarQube ships with an embedded H2 database for evaluation only. H2 does not support concurrent writes and cannot be backed up reliably. For any production or team use, PostgreSQL (or MSSQL/Oracle) is mandatory. The docker-compose setup above follows the production-recommended pattern with persistent volumes and strong passwords.

★ Best Practices

Configure Quality Gate on New Code only — prevents legacy debt from blocking all new development
Set branch protection rules in GitHub to require SonarQube Quality Gate pass before merge
Rotate SonarQube tokens every 90 days and use project-scoped tokens (not global analysis tokens) in CI
Enable security hotspot assignment — assign hotspots to responsible developers for review
Use SonarQube's branch analysis to see quality trends across feature branches
Export SonarQube metrics to Grafana/DataDog for executive-level security dashboards

⚠ Common Mistakes to Avoid

Using H2 embedded database in team environments — causes data loss and performance issues
Not setting `vm.max_map_count` — SonarQube's Elasticsearch component fails silently without it
Ignoring Security Hotspots because they are not auto-flagged as vulnerabilities
Not providing coverage reports — SonarQube shows 0% coverage without LCOV/JaCoCo/Cobertura files
Using admin/admin default credentials in any non-local environment

ASSIGNMENT 5 · SECRETS MANAGEMENT

Manage Secrets with HashiCorp Vault

Difficulty: ★★ ★ Advanced · Tools: HashiCorp Vault, Docker, GitHub Actions, AWS

Objective

Deploy HashiCorp Vault in development mode, configure the KV secrets engine, implement dynamic database credentials, and integrate with GitHub Actions using AppRole authentication.

Background & Theory

HashiCorp Vault is a secrets management platform that provides a unified interface to tokens, passwords, certificates, API keys, and encryption keys. It centralizes secrets with fine-grained access control, audit logging, and automatic secret rotation.

Dynamic secrets are Vault's killer feature — instead of storing a static database password shared by all applications, Vault generates a unique username and password for each service request, valid for a TTL (e.g., 1 hour). When the TTL expires, Vault automatically revokes the credentials. This eliminates credential sharing and drastically reduces the blast radius of a compromised secret.

Vault's authentication methods (AppRole, Kubernetes, AWS IAM, GitHub, LDAP) allow applications and CI/CD systems to authenticate to Vault without knowing a long-lived master password. AppRole is the recommended method for machine-to-machine authentication.

The Vault audit log records every interaction with Vault — who requested which secret, at what time, from which IP address. Combined with SIEM integration, this enables real-time detection of unusual secret access patterns.

Tasks

Task 1: Deploy Vault and configure KV secrets engine

- Run Vault in dev mode with Docker
- Enable and configure the KV v2 secrets engine
- Write and read secrets via CLI and API

Task 2: Configure AppRole authentication

- Enable AppRole auth method
- Create a policy for the application
- Generate RoleID and SecretID for GitHub Actions

Task 3: Dynamic database credentials

- Enable the database secrets engine
- Configure PostgreSQL connection
- Request dynamic credentials and verify TTL-based expiration

Complete Solution

Step 1: Deploy Vault with Docker

bash — Run Vault in dev mode

```
# Development mode (not for production — keys are in memory, unsealed automatically)
docker run -d --name vault \
  -p 8200:8200 \
  -e 'VAULT_DEV_ROOT_TOKEN_ID=devopsshack-root-token' \
  -e 'VAULT_DEV_LISTEN_ADDRESS=0.0.0.0:8200' \
  --cap-add=IPC_LOCK \
  hashicorp/vault:1.15

# Set environment variables
export VAULT_ADDR='http://127.0.0.1:8200'
export VAULT_TOKEN='devopsshack-root-token'

# Verify Vault is running
vault status
# Key          Value
# ---          -
# Seal Type    shamir
# Initialized   true
# Sealed       false
```

Step 2: KV Secrets Engine

bash — Enable and use KV v2 secrets engine

```
# Enable KV v2 (versioned secrets)
vault secrets enable -path=secret kv-v2

# Write a secret
vault kv put secret/devopsshack/app \
  db_password="vault_managed_password" \
  api_key="my-api-key-12345" \
  jwt_secret="super-secret-jwt-signing-key"

# Read the secret
vault kv get secret/devopsshack/app

# Read a specific field
vault kv get -field=db_password secret/devopsshack/app

# List secrets at a path
vault kv list secret/devopsshack/

# View version history
vault kv metadata get secret/devopsshack/app
```

```
# Roll back to previous version
vault kv rollback -version=1 secret/devopsshack/app

# Delete a secret version (soft delete, data still in metadata)
vault kv delete secret/devopsshack/app

# Permanently destroy a secret version
vault kv destroy -versions=2 secret/devopsshack/app
```

Step 3: Create Vault Policy

hcl — policies/app-policy.hcl

```
# Allow read access to application secrets
path "secret/data/devopsshack/app" {
  capabilities = ["read"]
}

# Allow listing secrets under devopsshack prefix
path "secret/metadata/devopsshack/*" {
  capabilities = ["list"]
}

# Allow requesting dynamic DB credentials
path "database/creds/app-role" {
  capabilities = ["read"]
}

# Deny access to all other paths
path "*" {
  capabilities = ["deny"]
}
```

bash — Apply policy

```
vault policy write app-policy policies/app-policy.hcl
vault policy list
vault policy read app-policy
```

Step 4: AppRole Authentication

bash — Configure AppRole for GitHub Actions

```
# Enable AppRole auth
vault auth enable approle

# Create role with policy attachment
vault write auth/approle/role/github-actions \
  token_policies="app-policy" \
  token_ttl=1h \
  token_max_ttl=4h \
  secret_id_ttl=24h \
```



```
secret_id_num_uses=10 # SecretID can be used max 10 times

# Get RoleID (store as GitHub Actions secret: VAULT_ROLE_ID)
vault read auth/approle/role/github-actions/role-id
# role_id = 988a9dfd-ea69-4a53-6cb6-9d6b86474bba

# Generate SecretID (store as GitHub Actions secret: VAULT_SECRET_ID)
vault write -f auth/approle/role/github-actions/secret-id
# secret_id = 37b74931-c4cd-d49a-9246-ccc62d682a25
```

Step 5: Dynamic Database Credentials

bash — Configure dynamic PostgreSQL credentials

```
# Enable database secrets engine
vault secrets enable database

# Configure PostgreSQL connection
vault write database/config/prod-postgres \
  plugin_name=postgresql-database-plugin \
  allowed_roles="app-role" \

connection_url="postgresql://{{username}}:{{password}}@postgres:5432/appdb?sslmode=disable" \
  username="vault_admin" \
  password="vault_admin_password"

# Create a role that generates credentials
vault write database/roles/app-role \
  db_name=prod-postgres \
  creation_statements="CREATE ROLE \"{{name}}\" WITH LOGIN PASSWORD '{{password}}' \
  VALID UNTIL '{{expiration}}'; \
  GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA public TO \"{{name}}\";" \
  default_ttl="1h" \
  max_ttl="24h"

# Request dynamic credentials
vault read database/creds/app-role
# Key          Value
# ---          ----
# lease_duration 1h
# username      v-approle-app-role-AbCdEf123456
# password      A1a-AbCdEfGhIjKl

# Renew lease before expiry
vault lease renew database/creds/app-role/LEASE_ID

# Revoke all credentials from a lease
vault lease revoke -prefix database/creds/app-role
```

Step 6: GitHub Actions Vault Integration

yml — .github/workflows/vault-secrets.yml

```
name: Deploy with Vault Secrets

on:
  push:
    branches: [ main ]

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Authenticate to Vault via AppRole
        uses: hashicorp/vault-action@v2
        with:
          url: ${ secrets.VAULT_ADDR }
          method: approle
          roleId: ${ secrets.VAULT_ROLE_ID }
          secretId: ${ secrets.VAULT_SECRET_ID }
          secrets: |
            secret/data/devopsshack/app db_password | DB_PASSWORD;
            secret/data/devopsshack/app api_key | API_KEY;
            secret/data/devopsshack/app jwt_secret | JWT_SECRET

      - name: Deploy application
        env:
          DB_PASSWORD: ${ env.DB_PASSWORD }
          API_KEY: ${ env.API_KEY }
        run: |
          echo "Deploying with Vault-managed secrets..."
          # Secrets are available as environment variables
          # They are masked in GitHub Actions logs automatically
```

Ultra-Deep Explanation

Vault's Secret Zero Problem and AppRole Solution

The "Secret Zero" problem: how does an application prove its identity to get its first secret without already having a secret? AppRole solves this by splitting the authentication credential into two parts: a RoleID (not sensitive, like a username — can be stored in code) and a SecretID (sensitive, like a password — delivered out-of-band via CI/CD platform secrets). Neither half alone can authenticate to Vault.

Dynamic Secrets: The Security Revolution

Traditional secrets are static and shared: every application instance uses the same database password, which never expires. If one instance is compromised, the attacker has permanent access. With dynamic secrets, each deployment gets unique credentials valid for 1 hour. When the TTL expires, the credentials are automatically revoked. The attacker's credentials expire within hours, not months.

Vault Seal and Unseal Mechanism

In production, Vault's master encryption key is split using Shamir's Secret Sharing into N key shards distributed among operators. To unseal Vault (decrypt its backend), M-of-N shards must be provided (e.g., 3-of-5). This means no single operator can unseal Vault alone, preventing insider threats. Auto-unseal using AWS KMS or Azure Key Vault eliminates manual intervention while maintaining security.

Audit Logging and Compliance

Every Vault API call is recorded in the audit log with: timestamp, caller identity, requested path, response status, and client IP. The audit log is append-only and cannot be disabled without root tokens (which themselves are audited). For PCI-DSS and SOC 2 compliance, Vault audit logs satisfy the requirement for "all access to cardholder data/secret material must be logged and monitored".

★ Best Practices

- Use Vault Agent for sidecar-based secret injection to avoid storing secrets in environment variables
- Configure seal/unseal with AWS KMS for production — eliminates manual key shards
- Set token_max_ttl to limit the lifetime of long-running jobs; use token renewal for services
- Enable Vault namespaces (Enterprise) or separate Vault clusters per environment
- Rotate root token immediately after initial setup — store it in a secure offline location
- Monitor Vault audit logs with SIEM for anomalous access patterns

⚠ Common Mistakes to Avoid

- Running Vault in dev mode in production — dev mode stores nothing on disk, all secrets lost on restart
- Using root token in CI/CD pipelines — create scoped AppRole tokens with minimal permissions
- Not setting token TTLs — long-lived tokens are equivalent to passwords that never expire
- Forgetting to renew dynamic credential leases — expired credentials cause application downtime
- Storing the SecretID in the same place as the RoleID — defeats the Secret Zero protection

ASSIGNMENT 6 · DAST

Dynamic Application Security Testing with OWASP ZAP

★ ★ Intermediate · Tools: OWASP ZAP, Docker, GitHub Actions

🎯 Objective

Deploy OWASP ZAP (Zed Attack Proxy) to perform dynamic application security testing (DAST) against a running web application, detecting OWASP Top 10 vulnerabilities including XSS, SQL injection, CSRF, and insecure headers.

📖 Background & Theory

DAST (Dynamic Application Security Testing) tests a running application from the outside, simulating an attacker's perspective. Unlike SAST which reads source code, DAST sends malicious payloads to endpoints and analyzes responses. This catches runtime vulnerabilities that static analysis misses.

OWASP ZAP (Zed Attack Proxy) is the world's most widely used web application security scanner, maintained by the Open Web Application Security Project (OWASP). It acts as a proxy between the browser and the application, intercepting and modifying HTTP traffic.

The OWASP Top 10 is the definitive list of the most critical web application security risks, updated every few years. The 2021 edition includes: Broken Access Control, Cryptographic Failures, Injection, Insecure Design, Security Misconfiguration, Vulnerable Components, Auth Failures, Integrity Failures, Security Logging Failures, and SSRF.

ZAP operates in two main modes: Passive Scan (observes traffic without modification) and Active Scan (actively tests endpoints with attack payloads). The Active Scanner should only be run against applications you own — it will attempt SQL injection, XSS, command injection, path traversal, and hundreds of other attacks.

📋 Tasks

Task 1: Deploy OWASP ZAP with Docker

- Pull and run the ZAP Docker image
- Access the ZAP web UI at port 8080
- Configure ZAP as a proxy for a test application

Task 2: Run automated baseline and full scan

- Run ZAP baseline scan against DVWA (Damn Vulnerable Web App)
- Execute active scan with attack strength High
- Generate HTML report from scan results

Task 3: Integrate ZAP into GitHub Actions

- Create workflow to spin up test application

- Run ZAP scan against the staged environment
- Parse ZAP JSON report and fail pipeline on HIGH findings

✓ Complete Solution

Step 1: Deploy Vulnerable Test Application (DVWA)

bash — Docker Compose for DVWA + ZAP

```
# docker-compose.yml
version: '3.8'
services:
  dvwa:
    image: vulnerables/web-dvwa:latest
    ports:
      - "8081:80"
    environment:
      - MYSQL_ROOT_PASSWORD=dvwa_root

  zap:
    image: ghcr.io/zaproxy/zaproxy:stable
    ports:
      - "8080:8080"
      - "8090:8090"
    command: zap-webswing.sh
    volumes:
      - zap-reports:/zap/wrk

volumes:
  zap-reports:

# Start services
docker-compose up -d

# Wait for DVWA to initialize
sleep 30

# Access DVWA: http://localhost:8081 (admin/password)
# Access ZAP UI: http://localhost:8080
```

Step 2: ZAP Baseline Scan (Passive)

bash — ZAP baseline and active scan

```
# Baseline scan (passive - safe for production-like environments)
docker run --rm -v $(pwd):/zap/wrk ghcr.io/zaproxy/zaproxy:stable \
  zap-baseline.py \
  -t http://dvwa:8081 \
  -r baseline-report.html \
  -J baseline-report.json \
  -w baseline-report.md \
  -l WARN # Minimum alert level: PASS, INFO, WARN, FAIL

# Full active scan (only run against apps you own!)
docker run --rm -v $(pwd):/zap/wrk ghcr.io/zaproxy/zaproxy:stable \
```

```
zap-full-scan.py \  
-t http://dvwa:8081 \  
-r full-scan-report.html \  
-J full-scan-report.json \  
-x full-scan-report.xml \  
-l HIGH \  
-z "-config scanner.attackStrength=HIGH"  
  
# API scan (using OpenAPI spec)  
docker run --rm -v $(pwd):/zap/wrk ghcr.io/zaproxy/zaproxy:stable \  
zap-api-scan.py \  
-t http://api:8082/openapi.json \  
-f openapi \  
-r api-scan-report.html
```

Step 3: Custom ZAP Rules Configuration

yaml — zap-rules.tsv (suppress false positives)

```
# Format: ruleId  IGNORE|WARN|FAIL  description  
10020  IGNORE  X-Frame-Options - handled by reverse proxy  
10021  IGNORE  X-Content-Type-Options - set at CDN  
10015  WARN     Incomplete or No Cache-control Header  
10038  FAIL     Content Security Policy (CSP) missing  
10010  FAIL     Cookie missing Secure flag
```

Step 4: GitHub Actions Integration

yaml — .github/workflows/dast.yml

```
name: DAST Security Scan  
  
on:  
  deployment_status:  
  push:  
    branches: [ develop ]  
  
jobs:  
  dast-zap-scan:  
    runs-on: ubuntu-latest  
    if: github.event.deployment_status.state == 'success' || github.ref ==  
    'refs/heads/develop'  
  
    steps:  
      - uses: actions/checkout@v4  
  
      - name: Deploy test application  
        run: docker-compose -f docker-compose.test.yml up -d && sleep 20  
  
      - name: ZAP Baseline Scan  
        uses: zaproxy/action-baseline@v0.12.0  
        with:  
          target: 'http://localhost:8081'  
          rules_file_name: 'zap-rules.tsv'  
          cmd_options: '-a'  
          issue_title: 'ZAP DAST Scan Report'
```

```
fail_action: true
allow_issue_writing: true

- name: Upload ZAP Report
  uses: actions/upload-artifact@v4
  if: always()
  with:
    name: zap-report
    path: report_html.html
```

Ultra-Deep Explanation

How ZAP's Active Scanner Works

ZAP's active scanner iterates through all discovered endpoints (gathered from spidering or passive proxying) and for each parameter, replaces the value with attack payloads. For SQL injection, it inserts characters like single quotes, double dashes, and UNION SELECT statements, then analyzes the response for error messages, timing differences, or unexpected content that indicates vulnerability.

OWASP Top 10 2021 — What ZAP Detects

A01 - Broken Access Control: ZAP tests path traversal, unauthorized direct object references, and privilege escalation. A02 - Cryptographic Failures: detects HTTP (non-HTTPS), weak cipher suites, missing HSTS headers. A03 - Injection: SQL, OS command, LDAP, XPath injection via active scanning. A07 - Identification and Authentication Failures: tests for default credentials, session token entropy, session fixation.

Authenticated Scanning

Many vulnerabilities only appear in authenticated sections of the application. ZAP supports form-based authentication (providing username/password to login form), HTTP basic/digest authentication, and cookie/token-based authentication. Configure a ZAP authentication script to log in before scanning to discover vulnerabilities in logged-in-only functionality.

Best Practices

Always run ZAP against a staging environment, never production — active scanning can cause data corruption

Use authenticated scanning to test access-controlled endpoints

Integrate ZAP scans into post-deployment pipeline stages, not during build

Combine ZAP (DAST) with SonarQube (SAST) and Trivy (SCA) for comprehensive coverage

Review false positives and tune rules file before enforcing hard failure

Common Mistakes to Avoid

Running active scan against production — it will attempt SQL injection and XSS

Not waiting for application startup before scanning — causes false negatives

Ignoring baseline scan results — missing headers are real security issues
Not scanning authenticated endpoints — leaves majority of application untested

ASSIGNMENT 7 · CONTAINER SECURITY**Kubernetes Security Policies with OPA/Gatekeeper**

★★★ Advanced · Tools: OPA, Gatekeeper, Kubernetes, Helm

Objective

Implement Open Policy Agent (OPA) Gatekeeper in Kubernetes to enforce security policies via admission control, preventing deployment of privileged containers, images from untrusted registries, and pods without resource limits.

Background & Theory

OPA (Open Policy Agent) is a general-purpose policy engine that decouples policy decisions from application code. In Kubernetes, OPA Gatekeeper acts as a validating admission webhook — it intercepts all API server requests (pod creation, deployment updates) and evaluates them against Rego policies before allowing or denying.

Rego is OPA's purpose-built policy language. It is a declarative logic language inspired by Datalog. Rego policies express rules as logical conditions: 'deny a pod if any container runs as root AND the pod is in the production namespace'. Rego is expressive, testable, and version-controllable.

Kubernetes admission controllers are plugins that intercept requests to the API server. There are two types: Mutating (can modify the object before it is stored) and Validating (can allow or deny the request). OPA Gatekeeper uses a Validating Admission Webhook — it evaluates the request and returns allow or deny.

Policy-as-code is the practice of defining and enforcing security and compliance policies through code (Rego, Python, HCL) rather than manual documentation. This enables automatic enforcement, version control, peer review, and audit trails — transforming compliance from a checkbox to a continuous automated process.

Tasks

Task 1: Install OPA Gatekeeper

- Deploy Gatekeeper using Helm
- Verify webhook configuration
- Understand ConstraintTemplate vs Constraint resources

Task 2: Write and test Rego policies

- Create ConstraintTemplate for privileged containers
- Create ConstraintTemplate for allowed image registries
- Write unit tests for Rego policies using OPA confest

Task 3: Apply constraints to namespaces

- Apply constraints to production namespace

- Test policy enforcement with intentionally violating pods
- Configure audit mode for existing violations

✓ Complete Solution

Step 1: Install OPA Gatekeeper

bash — Install Gatekeeper via Helm

```
# Add Gatekeeper Helm repo
helm repo add gatekeeper https://open-policy-agent.github.io/gatekeeper/charts
helm repo update

# Install Gatekeeper
helm install gatekeeper gatekeeper/gatekeeper \
  --namespace gatekeeper-system \
  --create-namespace \
  --set replicas=2 \
  --set auditInterval=60 \
  --set constraintViolationsLimit=100

# Verify installation
kubectl get pods -n gatekeeper-system
kubectl get validatingwebhookconfigurations
kubectl get constrainttemplates
```

Step 2: ConstraintTemplate - No Privileged Containers

yaml — constrainttemplate-noprivileged.yaml

```
apiVersion: templates.gatekeeper.sh/v1
kind: ConstraintTemplate
metadata:
  name: k8snoprivilegedcontainer
spec:
  crd:
    spec:
      names:
        kind: K8sNoPrivilegedContainer
  targets:
    - target: admission.k8s.gatekeeper.sh
      rego: |
        package k8snoprivilegedcontainer

        violation[{"msg": msg}] {
          container := input.review.object.spec.containers[_]
          container.securityContext.privileged == true
          msg := sprintf("Container <%v> in pod <%v> is privileged. Privileged
containers are not allowed.",
            [container.name, input.review.object.metadata.name])
        }

        violation[{"msg": msg}] {
          container := input.review.object.spec.initContainers[_]
          container.securityContext.privileged == true
        }
```

```

        msg := sprintf("Init container <%v> is privileged. Privileged containers
are not allowed.",
        [container.name])
    }

```

yaml — constraint-noprivileged.yaml

```

apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sNoPrivilegedContainer
metadata:
  name: no-privileged-containers
spec:
  enforcementAction: deny # Options: deny, dryrun, warn
  match:
    kinds:
      - apiGroups: [""]
        kinds: ["Pod"]
    namespaceSelector:
      matchLabels:
        environment: production

```

Step 3: ConstraintTemplate - Allowed Image Registries

yaml — constrainttemplate-allowedrepos.yaml

```

apiVersion: templates.gatekeeper.sh/v1
kind: ConstraintTemplate
metadata:
  name: k8sallowedrepos
spec:
  crd:
    spec:
      names:
        kind: K8sAllowedRepos
      validation:
        openAPIV3Schema:
          type: object
          properties:
            repos:
              type: array
              items:
                type: string
  targets:
    - target: admission.k8s.gatekeeper.sh
      rego: |
        package k8sallowedrepos

        violation[{"msg": msg}] {
          container := input.review.object.spec.containers[_]
          satisfied := [good | repo := input.parameters.repos[_]
                        good := startswith(container.image, repo)]

          not any(satisfied)
          msg := sprintf("Container <%v> uses image <%v> from a disallowed registry.
Allowed: %v",
          [container.name, container.image, input.parameters.repos])
        }
    ---
apiVersion: constraints.gatekeeper.sh/v1beta1

```

```
kind: K8sAllowedRepos
metadata:
  name: allowed-repos
spec:
  enforcementAction: deny
  match:
    kinds:
      - apiGroups: [""]
        kinds: ["Pod"]
  parameters:
    repos:
      - "ghcr.io/devopsshack/"
      - "public.ecr.aws/myorg/"
      - "gcr.io/myproject/"
```

Step 4: Resource Limits Required

yaml — constraint-resource-limits.yaml

```
apiVersion: templates.gatekeeper.sh/v1
kind: ConstraintTemplate
metadata:
  name: k8srequiredlimits
spec:
  crd:
    spec:
      names:
        kind: K8sRequiredLimits
  targets:
    - target: admission.k8s.gatekeeper.sh
      rego: |
        package k8srequiredlimits

        violation[{"msg": msg}] {
          container := input.review.object.spec.containers[_]
          not container.resources.limits.cpu
          msg := sprintf("Container <%v> has no CPU limit set.", [container.name])
        }

        violation[{"msg": msg}] {
          container := input.review.object.spec.containers[_]
          not container.resources.limits.memory
          msg := sprintf("Container <%v> has no memory limit set.",
[container.name])
        }
    ---
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sRequiredLimits
metadata:
  name: require-resource-limits
spec:
  enforcementAction: deny
  match:
    kinds:
      - apiGroups: ["apps"]
        kinds: ["Deployment", "DaemonSet", "StatefulSet"]
      - apiGroups: [""]
```

```
kinds: ["Pod"]
```

Ultra-Deep Explanation

Rego Language Deep Dive

Rego is a logic programming language. Rules are defined as logical implications: "violation is true if condition A AND condition B AND condition C are true". Gatekeeper calls the violation rule with the admission review object, and if any violation is generated, the request is denied with the violation message. The underscore `_` notation iterates over all elements of an array.

ConstraintTemplate vs Constraint Separation

ConstraintTemplate defines the policy logic (Rego) and the schema of the parameters it accepts. Constraint is an instance of a ConstraintTemplate, specifying which resources it applies to (match), what parameters to use, and the enforcement action. This separation enables reuse: one template for "image registry must be in allowed list", many constraints for different namespaces with different allowed registries.

Enforcement Actions: deny vs dryrun vs warn

deny immediately blocks the violating request. dryrun allows the request but records the violation in the constraint status — useful for auditing existing resources and implementing policies gradually. warn allows the request but returns a warning message to the user. The recommended rollout path is: dryrun (audit existing) → warn (alert without blocking) → deny (enforce).

★ Best Practices

- Start all new constraints in dryrun mode, audit violations, then move to warn then deny
- Write Rego unit tests using OPA's built-in test framework
- Label namespaces with environment=production to scope strict policies to production only
- Use Gatekeeper Audit to detect policy violations in resources that existed before the policy
- Version control all ConstraintTemplates and Constraints — they are code

⚠ Common Mistakes to Avoid

- Applying deny constraints to the gatekeeper-system namespace — causes Gatekeeper to block itself
- Writing overly broad constraints that block system components (kube-system, cert-manager)
- Not testing Rego policies with unit tests — logic errors cause unexpected denials
- Missing initContainers in privileged checks — attackers use init containers to escape

ASSIGNMENT 8 · NETWORK SECURITY

Network Policy Enforcement in Kubernetes

☆☆ Intermediate · Tools: Kubernetes, Calico, NetworkPolicy

Objective

Master Network Policy Enforcement in Kubernetes using Kubernetes, Calico, NetworkPolicy to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Network Security is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper network security controls expose themselves to significant security risks, compliance violations, and potential data breaches.

Kubernetes is a leading tool in the Network Security space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing network policy enforcement in kubernetes properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure Kubernetes

- Install Kubernetes using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure Kubernetes to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/kubernetes-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — Kubernetes installation

```
# Install Kubernetes
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "Kubernetes" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
kubernetes --version || kubernetes version

# Run initial health check
kubernetes --help

# Configure authentication if required
export KUBERNETES_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# kubernetes.yaml or .kubernetes.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: kubernetes-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yaml — .github/workflows/kubernetes.yml

```
name: Network Policy Enforcement in Kubernetes - Kubernetes

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Network Security Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run Kubernetes Scan
        run: |
          # Install and configure Kubernetes
          echo "Running Kubernetes security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: kubernetes-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## Kubernetes Security Scan Results\n See workflow artifacts
for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
kubernetes scan \
```



```

--severity CRITICAL,HIGH \
--format json \
--output results.json \
--exit-code 1 \
.

# Parse results programmatically
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:
  .FixedVersion}'

# Generate HTML report
kubernetes scan --format template \
  --template @html.tpl --output report.html .

# Fail pipeline only on CRITICAL
kubernetes scan --exit-code 1 --severity CRITICAL .

```

Step 5: Remediation Workflow

When Kubernetes finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```

# Create GitHub issue for each critical finding
cat results.json | jq -r '.Results[].Vulnerabilities[] |
  select(.Severity == "CRITICAL") |
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +
  " | Fix: " + (.FixedVersion // "No fix available")' | \
  while read line; do
    gh issue create --title "$line" --label "security,critical" --body "Auto-
generated from Kubernetes scan"
  done

# Track remediation metrics
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length')
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |
  select(.Severity=="CRITICAL")] | length')
echo "Total: $TOTAL | Critical: $CRITICAL"

```

Ultra-Deep Explanation

Kubernetes Architecture and Detection Engine

Kubernetes uses sophisticated analysis techniques to detect security issues in network security contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Network Security Security Principles

Network Security security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). Kubernetes implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin Kubernetes versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating Kubernetes's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 9 · CI/CD SECURITY

Secure GitHub Actions Workflows

★ ★ Intermediate · Tools: GitHub Actions, OIDC, CodeQL

Objective

Master Secure GitHub Actions Workflows using GitHub Actions, OIDC, CodeQL to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

CI/CD Security is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper ci/cd security controls expose themselves to significant security risks, compliance violations, and potential data breaches.

GitHub Actions is a leading tool in the CI/CD Security space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing secure github actions workflows properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure GitHub Actions

- Install GitHub Actions using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure GitHub Actions to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/github actions-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

☑ Complete Solution

Step 1: Installation and Initial Setup

bash — GitHub Actions installation

```
# Install GitHub Actions
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "GitHub Actions" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
github actions --version || github actions version

# Run initial health check
github actions --help

# Configure authentication if required
export GITHUB_ACTIONS_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# github actions.yaml or .github actions.yml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: github actions-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yml — .github/workflows/github actions.yml

```
name: Secure GitHub Actions Workflows - GitHub Actions

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: CI/CD Security Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run GitHub Actions Scan
        run: |
          # Install and configure GitHub Actions
          echo "Running GitHub Actions security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: github actions-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## GitHub Actions Security Scan Results\n See workflow
artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
github actions scan \
```

```

--severity CRITICAL,HIGH \
--format json \
--output results.json \
--exit-code 1 \
.

# Parse results programmatically
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:
  .FixedVersion}'

# Generate HTML report
github actions scan --format template \
  --template @html.tpl --output report.html .

# Fail pipeline only on CRITICAL
github actions scan --exit-code 1 --severity CRITICAL .

```

Step 5: Remediation Workflow

When GitHub Actions finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```

# Create GitHub issue for each critical finding
cat results.json | jq -r '.Results[].Vulnerabilities[] |
  select(.Severity == "CRITICAL") |
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +
  " | Fix: " + (.FixedVersion // "No fix available")' | \
  while read line; do
    gh issue create --title "$line" --label "security,critical" --body "Auto-
generated from GitHub Actions scan"
  done

# Track remediation metrics
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length')
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |
  select(.Severity=="CRITICAL")] | length')
echo "Total: $TOTAL | Critical: $CRITICAL"

```

Ultra-Deep Explanation

GitHub Actions Architecture and Detection Engine

GitHub Actions uses sophisticated analysis techniques to detect security issues in ci/cd security contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: CI/CD Security Security Principles

CI/CD Security security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). GitHub Actions implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin GitHub Actions versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating GitHub Actions's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 10 · COMPLIANCE

Kubernetes CIS Benchmark with kube-bench

★ ★ Intermediate · Tools: kube-bench, Kubernetes, AWS EKS

Objective

Master Kubernetes CIS Benchmark with kube-bench using kube-bench, Kubernetes, AWS EKS to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Compliance is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper compliance controls expose themselves to significant security risks, compliance violations, and potential data breaches.

kube-bench is a leading tool in the Compliance space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing kubernetes cis benchmark with kube-bench properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure kube-bench

- Install kube-bench using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure kube-bench to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/kube-bench-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — kube-bench installation

```
# Install kube-bench
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "kube-bench" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
kube-bench --version || kube-bench version

# Run initial health check
kube-bench --help

# Configure authentication if required
export KUBE_BENCH_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# kube-bench.yaml or .kube-bench.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: kube-bench-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yaml — .github/workflows/kube-bench.yml

```
name: Kubernetes CIS Benchmark with kube-bench - kube-bench

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Compliance Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run kube-bench Scan
        run: |
          # Install and configure kube-bench
          echo "Running kube-bench security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: kube-bench-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## kube-bench Security Scan Results\n See workflow artifacts
for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
kube-bench scan \
```

```
--severity CRITICAL,HIGH \  
--format json \  
--output results.json \  
--exit-code 1 \  
.  
  
# Parse results programmatically  
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |  
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |  
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:  
  .FixedVersion}'  
  
# Generate HTML report  
kube-bench scan --format template \  
  --template @html.tpl --output report.html .  
  
# Fail pipeline only on CRITICAL  
kube-bench scan --exit-code 1 --severity CRITICAL .
```

Step 5: Remediation Workflow

When kube-bench finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```
# Create GitHub issue for each critical finding  
cat results.json | jq -r '.Results[].Vulnerabilities[] |  
  select(.Severity == "CRITICAL") |  
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +  
  " | Fix: " + (.FixedVersion // "No fix available")' | \  
while read line; do  
  gh issue create --title "$line" --label "security,critical" --body "Auto-  
generated from kube-bench scan"  
done  
  
# Track remediation metrics  
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length']  
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |  
  select(.Severity=="CRITICAL")] | length')  
echo "Total: $TOTAL | Critical: $CRITICAL"
```

Ultra-Deep Explanation

kube-bench Architecture and Detection Engine

kube-bench uses sophisticated analysis techniques to detect security issues in compliance contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Compliance Security Principles

Compliance security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). kube-bench implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin kube-bench versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating kube-bench's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 11 · RUNTIME SECURITY

Runtime Threat Detection with Falco

☆☆☆ Advanced · Tools: Falco, Kubernetes, Slack Alerts

Objective

Master Runtime Threat Detection with Falco using Falco, Kubernetes, Slack Alerts to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Runtime Security is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper runtime security controls expose themselves to significant security risks, compliance violations, and potential data breaches.

Falco is a leading tool in the Runtime Security space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing runtime threat detection with falco properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure Falco

- Install Falco using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure Falco to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/falco-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

☑ Complete Solution

Step 1: Installation and Initial Setup

bash — Falco installation

```
# Install Falco
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "Falco" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
falco --version || falco version

# Run initial health check
falco --help

# Configure authentication if required
export FALCO_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# falco.yaml or .falco.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: falco-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yaml — .github/workflows/falco.yml

```
name: Runtime Threat Detection with Falco - Falco

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Runtime Security Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run Falco Scan
        run: |
          # Install and configure Falco
          echo "Running Falco security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: falco-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## Falco Security Scan Results\n See workflow artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
falco scan \
```

```

--severity CRITICAL,HIGH \
--format json \
--output results.json \
--exit-code 1 \
.

# Parse results programmatically
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:
  .FixedVersion}'

# Generate HTML report
falco scan --format template \
  --template @html.tpl --output report.html .

# Fail pipeline only on CRITICAL
falco scan --exit-code 1 --severity CRITICAL .

```

Step 5: Remediation Workflow

When Falco finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```

# Create GitHub issue for each critical finding
cat results.json | jq -r '.Results[].Vulnerabilities[] |
  select(.Severity == "CRITICAL") |
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +
  " | Fix: " + (.FixedVersion // "No fix available")' | \
while read line; do
  gh issue create --title "$line" --label "security,critical" --body "Auto-
generated from Falco scan"
done

# Track remediation metrics
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length')
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |
select(.Severity=="CRITICAL")] | length')
echo "Total: $TOTAL | Critical: $CRITICAL"

```

Ultra-Deep Explanation

Falco Architecture and Detection Engine

Falco uses sophisticated analysis techniques to detect security issues in runtime security contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Runtime Security Security Principles

Runtime Security security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). Falco implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin Falco versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating Falco's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 12 · SUPPLY CHAIN

SBOM Generation and Verification

★ ★ Intermediate · Tools: Syft, Gype, CycloneDX, SPDX

Objective

Master SBOM Generation and Verification using Syft, Gype, CycloneDX, SPDX to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Supply Chain is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper supply chain controls expose themselves to significant security risks, compliance violations, and potential data breaches.

Syft is a leading tool in the Supply Chain space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing sbom generation and verification properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure Syft

- Install Syft using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure Syft to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/syft-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

✓ Complete Solution

Step 1: Installation and Initial Setup

bash — Syft installation

```
# Install Syft
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "Syft" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
syft --version || syft version

# Run initial health check
syft --help

# Configure authentication if required
export SYFT_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# syft.yaml or .syft.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: syft-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yaml — .github/workflows/syft.yml

```
name: SBOM Generation and Verification - Syft

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Supply Chain Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run Syft Scan
        run: |
          # Install and configure Syft
          echo "Running Syft security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: syft-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## Syft Security Scan Results\n See workflow artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
syft scan \
```



```

--severity CRITICAL,HIGH \
--format json \
--output results.json \
--exit-code 1 \
.

# Parse results programmatically
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:
  .FixedVersion}'

# Generate HTML report
syft scan --format template \
  --template @html.tpl --output report.html .

# Fail pipeline only on CRITICAL
syft scan --exit-code 1 --severity CRITICAL .

```

Step 5: Remediation Workflow

When Syft finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```

# Create GitHub issue for each critical finding
cat results.json | jq -r '.Results[].Vulnerabilities[] |
  select(.Severity == "CRITICAL") |
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +
  " | Fix: " + (.FixedVersion // "No fix available")' | \
while read line; do
  gh issue create --title "$line" --label "security,critical" --body "Auto-
generated from Syft scan"
done

# Track remediation metrics
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length')
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |
select(.Severity=="CRITICAL")] | length')
echo "Total: $TOTAL | Critical: $CRITICAL"

```

Ultra-Deep Explanation

Syft Architecture and Detection Engine

Syft uses sophisticated analysis techniques to detect security issues in supply chain contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Supply Chain Security Principles

Supply Chain security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). Syft implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin Syft versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating Syft's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 13 · IDENTITY & ACCESS

Kubernetes RBAC Deep Dive

★ ★ Intermediate · Tools: Kubernetes, kubectl, RBAC

🎯 Objective

Master Kubernetes RBAC Deep Dive using Kubernetes, kubectl, RBAC to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

📖 Background & Theory

Identity & Access is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper identity & access controls expose themselves to significant security risks, compliance violations, and potential data breaches.

Kubernetes is a leading tool in the Identity & Access space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing kubernetes rbac deep dive properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

📋 Tasks

Task 1: Install and Configure Kubernetes

- Install Kubernetes using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure Kubernetes to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/kubernetes-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — Kubernetes installation

```
# Install Kubernetes
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "Kubernetes" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
kubernetes --version || kubernetes version

# Run initial health check
kubernetes --help

# Configure authentication if required
export KUBERNETES_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# kubernetes.yaml or .kubernetes.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: kubernetes-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yaml — .github/workflows/kubernetes.yml

```
name: Kubernetes RBAC Deep Dive - Kubernetes

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Identity & Access Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run Kubernetes Scan
        run: |
          # Install and configure Kubernetes
          echo "Running Kubernetes security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: kubernetes-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## Kubernetes Security Scan Results\n See workflow artifacts
for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
kubernetes scan \
```

```

--severity CRITICAL,HIGH \
--format json \
--output results.json \
--exit-code 1 \
.

# Parse results programmatically
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:
  .FixedVersion}'

# Generate HTML report
kubernetes scan --format template \
  --template @html.tpl --output report.html .

# Fail pipeline only on CRITICAL
kubernetes scan --exit-code 1 --severity CRITICAL .

```

Step 5: Remediation Workflow

When Kubernetes finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```

# Create GitHub issue for each critical finding
cat results.json | jq -r '.Results[].Vulnerabilities[] |
  select(.Severity == "CRITICAL") |
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +
  " | Fix: " + (.FixedVersion // "No fix available")' | \
  while read line; do
    gh issue create --title "$line" --label "security,critical" --body "Auto-
generated from Kubernetes scan"
  done

# Track remediation metrics
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length')
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |
  select(.Severity=="CRITICAL")] | length')
echo "Total: $TOTAL | Critical: $CRITICAL"

```

Ultra-Deep Explanation

Kubernetes Architecture and Detection Engine

Kubernetes uses sophisticated analysis techniques to detect security issues in identity & access contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Identity & Access Security Principles

Identity & Access security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). Kubernetes implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin Kubernetes versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating Kubernetes's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 14 · SECRETS MANAGEMENT**External Secrets Operator with AWS Secrets Manager**

★ ★ ★ Advanced · Tools: External Secrets Operator, AWS, Kubernetes

Objective

Master External Secrets Operator with AWS Secrets Manager using External Secrets Operator, AWS, Kubernetes to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Secrets Management is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper secrets management controls expose themselves to significant security risks, compliance violations, and potential data breaches.

External Secrets Operator is a leading tool in the Secrets Management space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing external secrets operator with aws secrets manager properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure External Secrets Operator

- Install External Secrets Operator using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure External Secrets Operator to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create .github/workflows/external secrets operator-scan.yml
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

✓ Complete Solution

Step 1: Installation and Initial Setup

bash — External Secrets Operator installation

```
# Install External Secrets Operator
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "External Secrets Operator" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
external secrets operator --version || external secrets operator version

# Run initial health check
external secrets operator --help

# Configure authentication if required
export EXTERNAL_SECRETS_OPERATOR_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# external secrets operator.yaml or .external secrets operator.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: external secrets operator-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yml — .github/workflows/external secrets operator.yml

```
name: External Secrets Operator with AWS Secrets Manager - External Secrets Operator

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Secrets Management Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run External Secrets Operator Scan
        run: |
          # Install and configure External Secrets Operator
          echo "Running External Secrets Operator security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: external-secrets-operator-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## External Secrets Operator Security Scan Results\n See workflow artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
external-secrets-operator scan \
```

```

--severity CRITICAL,HIGH \
--format json \
--output results.json \
--exit-code 1 \
.

# Parse results programmatically
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:
  .FixedVersion}'

# Generate HTML report
external secrets operator scan --format template \
  --template @html.tpl --output report.html .

# Fail pipeline only on CRITICAL
external secrets operator scan --exit-code 1 --severity CRITICAL .

```

Step 5: Remediation Workflow

When External Secrets Operator finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```

# Create GitHub issue for each critical finding
cat results.json | jq -r '.Results[].Vulnerabilities[] |
  select(.Severity == "CRITICAL") |
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +
  " | Fix: " + (.FixedVersion // "No fix available")' | \
while read line; do
  gh issue create --title "$line" --label "security,critical" --body "Auto-
generated from External Secrets Operator scan"
done

# Track remediation metrics
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length')
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |
  select(.Severity=="CRITICAL")] | length')
echo "Total: $TOTAL | Critical: $CRITICAL"

```

Ultra-Deep Explanation

External Secrets Operator Architecture and Detection Engine

External Secrets Operator uses sophisticated analysis techniques to detect security issues in secrets management contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while

managing false positive rates. Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Secrets Management Security Principles

Secrets Management security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). External Secrets Operator implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin External Secrets Operator versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating External Secrets Operator's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 15 · IAC SECURITY

Terraform State Security and Remote Backends

★ ★ Intermediate · Tools: Terraform, S3, DynamoDB, KMS

🎯 Objective

Master Terraform State Security and Remote Backends using Terraform, S3, DynamoDB, KMS to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

📖 Background & Theory

IaC Security is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper iac security controls expose themselves to significant security risks, compliance violations, and potential data breaches.

Terraform is a leading tool in the IaC Security space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing terraform state security and remote backends properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

📋 Tasks

Task 1: Install and Configure Terraform

- Install Terraform using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure Terraform to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/terraform-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — Terraform installation

```
# Install Terraform
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "Terraform" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
terraform --version || terraform version

# Run initial health check
terraform --help

# Configure authentication if required
export TERRAFORM_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# terraform.yaml or .terraform.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: terraform-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yml — .github/workflows/terraform.yml

```
name: Terraform State Security and Remote Backends - Terraform

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: IaC Security Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run Terraform Scan
        run: |
          # Install and configure Terraform
          echo "Running Terraform security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: terraform-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## Terraform Security Scan Results\n See workflow artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
terraform scan \
```

```

--severity CRITICAL,HIGH \
--format json \
--output results.json \
--exit-code 1 \
.

# Parse results programmatically
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:
  .FixedVersion}'

# Generate HTML report
terraform scan --format template \
  --template @html.tpl --output report.html .

# Fail pipeline only on CRITICAL
terraform scan --exit-code 1 --severity CRITICAL .

```

Step 5: Remediation Workflow

When Terraform finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```

# Create GitHub issue for each critical finding
cat results.json | jq -r '.Results[].Vulnerabilities[] |
  select(.Severity == "CRITICAL") |
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +
  " | Fix: " + (.FixedVersion // "No fix available")' | \
  while read line; do
    gh issue create --title "$line" --label "security,critical" --body "Auto-
generated from Terraform scan"
  done

# Track remediation metrics
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length')
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |
  select(.Severity=="CRITICAL")] | length')
echo "Total: $TOTAL | Critical: $CRITICAL"

```

Ultra-Deep Explanation

Terraform Architecture and Detection Engine

Terraform uses sophisticated analysis techniques to detect security issues in iac security contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: IaC Security Principles

IaC Security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). Terraform implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin Terraform versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating Terraform's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 16 · IMAGE SECURITY

Signing Container Images with Cosign

★ ★ ★ Advanced · Tools: Cosign, GHCR, Sigstore, Rekor

Objective

Master Signing Container Images with Cosign using Cosign, GHCR, Sigstore, Rekor to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Image Security is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper image security controls expose themselves to significant security risks, compliance violations, and potential data breaches.

Cosign is a leading tool in the Image Security space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing signing container images with cosign properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure Cosign

- Install Cosign using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure Cosign to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/cosign-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

☑ Complete Solution

Step 1: Installation and Initial Setup

bash — Cosign installation

```
# Install Cosign
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "Cosign" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
cosign --version || cosign version

# Run initial health check
cosign --help

# Configure authentication if required
export COSIGN_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# cosign.yaml or .cosign.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: cosign-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yaml — .github/workflows/cosign.yml

```
name: Signing Container Images with Cosign - Cosign

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Image Security Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run Cosign Scan
        run: |
          # Install and configure Cosign
          echo "Running Cosign security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: cosign-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## Cosign Security Scan Results\n See workflow artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
cosign scan \
```



```

--severity CRITICAL,HIGH \
--format json \
--output results.json \
--exit-code 1 \
.

# Parse results programmatically
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:
  .FixedVersion}'

# Generate HTML report
cosign scan --format template \
  --template @html.tpl --output report.html .

# Fail pipeline only on CRITICAL
cosign scan --exit-code 1 --severity CRITICAL .

```

Step 5: Remediation Workflow

When Cosign finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```

# Create GitHub issue for each critical finding
cat results.json | jq -r '.Results[].Vulnerabilities[] |
  select(.Severity == "CRITICAL") |
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +
  " | Fix: " + (.FixedVersion // "No fix available")' | \
while read line; do
  gh issue create --title "$line" --label "security,critical" --body "Auto-
generated from Cosign scan"
done

# Track remediation metrics
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length']
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |
select(.Severity=="CRITICAL")] | length')
echo "Total: $TOTAL | Critical: $CRITICAL"

```

Ultra-Deep Explanation

Cosign Architecture and Detection Engine

Cosign uses sophisticated analysis techniques to detect security issues in image security contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Image Security Security Principles

Image Security security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). Cosign implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin Cosign versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating Cosign's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 17 · MONITORING

Security Monitoring with Prometheus and Grafana

★ ★ Intermediate · Tools: Prometheus, Grafana, AlertManager

Objective

Master Security Monitoring with Prometheus and Grafana using Prometheus, Grafana, AlertManager to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Monitoring is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper monitoring controls expose themselves to significant security risks, compliance violations, and potential data breaches.

Prometheus is a leading tool in the Monitoring space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing security monitoring with prometheus and grafana properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure Prometheus

- Install Prometheus using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure Prometheus to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/prometheus-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

☑ Complete Solution

Step 1: Installation and Initial Setup

bash — Prometheus installation

```
# Install Prometheus
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "Prometheus" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
prometheus --version || prometheus version

# Run initial health check
prometheus --help

# Configure authentication if required
export PROMETHEUS_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# prometheus.yaml or .prometheus.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: prometheus-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yml — .github/workflows/prometheus.yml

```
name: Security Monitoring with Prometheus and Grafana - Prometheus

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Monitoring Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run Prometheus Scan
        run: |
          # Install and configure Prometheus
          echo "Running Prometheus security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: prometheus-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## Prometheus Security Scan Results\n See workflow artifacts
for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
prometheus scan \
```

```
--severity CRITICAL,HIGH \
--format json \
--output results.json \
--exit-code 1 \
.

# Parse results programmatically
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:
  .FixedVersion}'

# Generate HTML report
prometheus scan --format template \
  --template @html.tpl --output report.html .

# Fail pipeline only on CRITICAL
prometheus scan --exit-code 1 --severity CRITICAL .
```

Step 5: Remediation Workflow

When Prometheus finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```
# Create GitHub issue for each critical finding
cat results.json | jq -r '.Results[].Vulnerabilities[] |
  select(.Severity == "CRITICAL") |
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +
  " | Fix: " + (.FixedVersion // "No fix available")' | \
  while read line; do
    gh issue create --title "$line" --label "security,critical" --body "Auto-
generated from Prometheus scan"
  done

# Track remediation metrics
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length')
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |
  select(.Severity=="CRITICAL")] | length')
echo "Total: $TOTAL | Critical: $CRITICAL"
```

Ultra-Deep Explanation

Prometheus Architecture and Detection Engine

Prometheus uses sophisticated analysis techniques to detect security issues in monitoring contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Monitoring Security Principles

Monitoring security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). Prometheus implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin Prometheus versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating Prometheus's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 18 · VULNERABILITY MANAGEMENT

CVE Management with Dependency Track

☆☆☆ Advanced · Tools: OWASP Dependency-Track, CycloneDX

Objective

Master CVE Management with Dependency Track using OWASP Dependency-Track, CycloneDX to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Vulnerability Management is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper vulnerability management controls expose themselves to significant security risks, compliance violations, and potential data breaches.

OWASP Dependency-Track is a leading tool in the Vulnerability Management space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing cve management with dependency track properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure OWASP Dependency-Track

- Install OWASP Dependency-Track using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure OWASP Dependency-Track to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/owasp dependency-track-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — OWASP Dependency-Track installation

```
# Install OWASP Dependency-Track
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "OWASP Dependency-Track" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
owasp dependency-track --version || owasp dependency-track version

# Run initial health check
owasp dependency-track --help

# Configure authentication if required
export OWASP_DEPENDENCY_TRACK_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# owasp dependency-track.yml or .owasp dependency-track.yml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: owasp dependency-track-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yml — .github/workflows/owasp dependency-track.yml

```
name: CVE Management with Dependency Track - OWASP Dependency-Track

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Vulnerability Management Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run OWASP Dependency-Track Scan
        run: |
          # Install and configure OWASP Dependency-Track
          echo "Running OWASP Dependency-Track security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: owasp dependency-track-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## OWASP Dependency-Track Security Scan Results\n See workflow artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
owasp dependency-track scan \
```

```

--severity CRITICAL,HIGH \
--format json \
--output results.json \
--exit-code 1 \
.

# Parse results programmatically
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:
  .FixedVersion}'

# Generate HTML report
owasp dependency-track scan --format template \
  --template @html.tpl --output report.html .

# Fail pipeline only on CRITICAL
owasp dependency-track scan --exit-code 1 --severity CRITICAL .

```

Step 5: Remediation Workflow

When OWASP Dependency-Track finds security issues, the remediation workflow should be:

(1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```

# Create GitHub issue for each critical finding
cat results.json | jq -r '.Results[].Vulnerabilities[] |
  select(.Severity == "CRITICAL") |
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +
  " | Fix: " + (.FixedVersion // "No fix available")' | \
  while read line; do
    gh issue create --title "$line" --label "security,critical" --body "Auto-
generated from OWASP Dependency-Track scan"
  done

# Track remediation metrics
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length'] )
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |
  select(.Severity=="CRITICAL")] | length')
echo "Total: $TOTAL | Critical: $CRITICAL"

```

Ultra-Deep Explanation

OWASP Dependency-Track Architecture and Detection Engine

OWASP Dependency-Track uses sophisticated analysis techniques to detect security issues in vulnerability management contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while

managing false positive rates. Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Vulnerability Management Security Principles

Vulnerability Management security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). OWASP Dependency-Track implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin OWASP Dependency-Track versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating OWASP Dependency-Track's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 19 · NETWORK SECURITY

Service Mesh Security with Istio mTLS

★ ★ ★ Advanced · Tools: Istio, Kubernetes, mTLS, SPIFFE

Objective

Master Service Mesh Security with Istio mTLS using Istio, Kubernetes, mTLS, SPIFFE to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Network Security is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper network security controls expose themselves to significant security risks, compliance violations, and potential data breaches.

Istio is a leading tool in the Network Security space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing service mesh security with istio mtls properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure Istio

- Install Istio using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure Istio to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/istio-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

☑ Complete Solution

Step 1: Installation and Initial Setup

bash — Istio installation

```
# Install Istio
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "Istio" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
istio --version || istio version

# Run initial health check
istio --help

# Configure authentication if required
export ISTIO_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# istio.yaml or .istio.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: istio-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yml — .github/workflows/istio.yml

```
name: Service Mesh Security with Istio mTLS - Istio

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Network Security Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run Istio Scan
        run: |
          # Install and configure Istio
          echo "Running Istio security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: istio-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## Istio Security Scan Results\n See workflow artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
istio scan \
```

```

--severity CRITICAL,HIGH \
--format json \
--output results.json \
--exit-code 1 \
.

# Parse results programmatically
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:
  .FixedVersion}'

# Generate HTML report
istio scan --format template \
  --template @html.tpl --output report.html .

# Fail pipeline only on CRITICAL
istio scan --exit-code 1 --severity CRITICAL .

```

Step 5: Remediation Workflow

When Istio finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```

# Create GitHub issue for each critical finding
cat results.json | jq -r '.Results[].Vulnerabilities[] |
  select(.Severity == "CRITICAL") |
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +
  " | Fix: " + (.FixedVersion // "No fix available")' | \
while read line; do
  gh issue create --title "$line" --label "security,critical" --body "Auto-
generated from Istio scan"
done

# Track remediation metrics
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length')
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |
select(.Severity=="CRITICAL")] | length')
echo "Total: $TOTAL | Critical: $CRITICAL"

```

Ultra-Deep Explanation

Istio Architecture and Detection Engine

Istio uses sophisticated analysis techniques to detect security issues in network security contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Network Security Security Principles

Network Security security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). Istio implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin Istio versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating Istio's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 20 · COMPLIANCE

Cloud Security Posture Management with AWS SecurityHub

★ ★ Intermediate · Tools: AWS SecurityHub, GuardDuty, Config

🎯 Objective

Master Cloud Security Posture Management with AWS SecurityHub using AWS SecurityHub, GuardDuty, Config to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

📖 Background & Theory

Compliance is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper compliance controls expose themselves to significant security risks, compliance violations, and potential data breaches.

AWS SecurityHub is a leading tool in the Compliance space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing cloud security posture management with aws securityhub properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

📋 Tasks

Task 1: Install and Configure AWS SecurityHub

- Install AWS SecurityHub using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure AWS SecurityHub to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/aws securityhub-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — AWS SecurityHub installation

```
# Install AWS SecurityHub
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "AWS SecurityHub" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
aws securityhub --version || aws securityhub version

# Run initial health check
aws securityhub --help

# Configure authentication if required
export AWS_SECURITYHUB_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# aws securityhub.yaml or .aws securityhub.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: aws securityhub-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yaml — .github/workflows/aws securityhub.yml

```
name: Cloud Security Posture Management with AWS SecurityHub - AWS SecurityHub

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Compliance Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run AWS SecurityHub Scan
        run: |
          # Install and configure AWS SecurityHub
          echo "Running AWS SecurityHub security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: aws securityhub-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## AWS SecurityHub Security Scan Results\n See workflow
artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options


```
# Scan with additional context
aws securityhub scan \
  --severity CRITICAL,HIGH \
  --format json \
  --output results.json \
  --exit-code 1 \
  .

# Parse results programmatically
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:
  .FixedVersion}'

# Generate HTML report
aws securityhub scan --format template \
  --template @html.tpl --output report.html .

# Fail pipeline only on CRITICAL
aws securityhub scan --exit-code 1 --severity CRITICAL .
```

Step 5: Remediation Workflow

When AWS SecurityHub finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```
# Create GitHub issue for each critical finding
cat results.json | jq -r '.Results[].Vulnerabilities[] |
  select(.Severity == "CRITICAL") |
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +
  " | Fix: " + (.FixedVersion // "No fix available")' | \
while read line; do
  gh issue create --title "$line" --label "security,critical" --body "Auto-
generated from AWS SecurityHub scan"
done

# Track remediation metrics
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] ] | length')
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |
select(.Severity=="CRITICAL")] | length')
echo "Total: $TOTAL | Critical: $CRITICAL"
```

Ultra-Deep Explanation

AWS SecurityHub Architecture and Detection Engine

AWS SecurityHub uses sophisticated analysis techniques to detect security issues in compliance contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates. Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Compliance Security Principles

Compliance security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). AWS SecurityHub implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin AWS SecurityHub versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating AWS SecurityHub's vulnerability database before scanning

Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 21 · CONTAINER SECURITY

Pod Security Standards and Admission Control

★ ★ Intermediate · Tools: Kubernetes PSS, OPA, Admission Webhooks

Objective

Master Pod Security Standards and Admission Control using Kubernetes PSS, OPA, Admission Webhooks to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Container Security is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper container security controls expose themselves to significant security risks, compliance violations, and potential data breaches.

Kubernetes PSS is a leading tool in the Container Security space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing pod security standards and admission control properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure Kubernetes PSS

- Install Kubernetes PSS using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure Kubernetes PSS to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/kubernetes-pss-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

☑ Complete Solution

Step 1: Installation and Initial Setup

bash — Kubernetes PSS installation

```
# Install Kubernetes PSS
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "Kubernetes PSS" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
kubernetes pss --version || kubernetes pss version

# Run initial health check
kubernetes pss --help

# Configure authentication if required
export KUBERNETES_PSS_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# kubernetes pss.yaml or .kubernetes pss.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: kubernetes-pss-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yaml — .github/workflows/kubernetes pss.yml

```
name: Pod Security Standards and Admission Control - Kubernetes PSS

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Container Security Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run Kubernetes PSS Scan
        run: |
          # Install and configure Kubernetes PSS
          echo "Running Kubernetes PSS security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: kubernetes pss-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## Kubernetes PSS Security Scan Results\n See workflow
artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
kubernetes pss scan \
```

```

--severity CRITICAL,HIGH \
--format json \
--output results.json \
--exit-code 1 \
.

# Parse results programmatically
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:
  .FixedVersion}'

# Generate HTML report
kubernetes pss scan --format template \
  --template @html.tpl --output report.html .

# Fail pipeline only on CRITICAL
kubernetes pss scan --exit-code 1 --severity CRITICAL .

```

Step 5: Remediation Workflow

When Kubernetes PSS finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```

# Create GitHub issue for each critical finding
cat results.json | jq -r '.Results[].Vulnerabilities[] |
  select(.Severity == "CRITICAL") |
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +
  " | Fix: " + (.FixedVersion // "No fix available")' | \
  while read line; do
    gh issue create --title "$line" --label "security,critical" --body "Auto-
generated from Kubernetes PSS scan"
  done

# Track remediation metrics
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length')
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |
  select(.Severity=="CRITICAL")] | length')
echo "Total: $TOTAL | Critical: $CRITICAL"

```

Ultra-Deep Explanation

Kubernetes PSS Architecture and Detection Engine

Kubernetes PSS uses sophisticated analysis techniques to detect security issues in container security contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false

positive rates. Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Container Security Security Principles

Container Security security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). Kubernetes PSS implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin Kubernetes PSS versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating Kubernetes PSS's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 22 · CI/CD SECURITY

Jenkins Pipeline Security Hardening

★ ★ Intermediate · Tools: Jenkins, Credentials Store, Pipeline

Objective

Master Jenkins Pipeline Security Hardening using Jenkins, Credentials Store, Pipeline to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

CI/CD Security is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper ci/cd security controls expose themselves to significant security risks, compliance violations, and potential data breaches.

Jenkins is a leading tool in the CI/CD Security space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing jenkins pipeline security hardening properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure Jenkins

- Install Jenkins using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure Jenkins to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/jenkins-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — Jenkins installation

```
# Install Jenkins
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "Jenkins" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
jenkins --version || jenkins version

# Run initial health check
jenkins --help

# Configure authentication if required
export JENKINS_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# jenkins.yaml or .jenkins.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: jenkins-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yml — .github/workflows/jenkins.yml

```
name: Jenkins Pipeline Security Hardening - Jenkins

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: CI/CD Security Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run Jenkins Scan
        run: |
          # Install and configure Jenkins
          echo "Running Jenkins security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: jenkins-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## Jenkins Security Scan Results\n See workflow artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
jenkins scan \
```

```

--severity CRITICAL,HIGH \
--format json \
--output results.json \
--exit-code 1 \
.

# Parse results programmatically
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:
  .FixedVersion}'

# Generate HTML report
jenkins scan --format template \
  --template @html.tpl --output report.html .

# Fail pipeline only on CRITICAL
jenkins scan --exit-code 1 --severity CRITICAL .

```

Step 5: Remediation Workflow

When Jenkins finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```

# Create GitHub issue for each critical finding
cat results.json | jq -r '.Results[].Vulnerabilities[] |
  select(.Severity == "CRITICAL") |
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +
  " | Fix: " + (.FixedVersion // "No fix available")' | \
  while read line; do
    gh issue create --title "$line" --label "security,critical" --body "Auto-
generated from Jenkins scan"
  done

# Track remediation metrics
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length')
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |
  select(.Severity=="CRITICAL")] | length')
echo "Total: $TOTAL | Critical: $CRITICAL"

```

Ultra-Deep Explanation

Jenkins Architecture and Detection Engine

Jenkins uses sophisticated analysis techniques to detect security issues in ci/cd security contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: CI/CD Security Security Principles

CI/CD Security security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). Jenkins implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin Jenkins versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating Jenkins's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 23 · CRYPTOGRAPHY**TLS Certificate Management with cert-manager**

☆☆ Intermediate · Tools: cert-manager, Let's Encrypt, Kubernetes

Objective

Master TLS Certificate Management with cert-manager using cert-manager, Let's Encrypt, Kubernetes to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Cryptography is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper cryptography controls expose themselves to significant security risks, compliance violations, and potential data breaches.

cert-manager is a leading tool in the Cryptography space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing tls certificate management with cert-manager properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure cert-manager

- Install cert-manager using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure cert-manager to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/cert-manager-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

✅ Complete Solution

Step 1: Installation and Initial Setup

bash — cert-manager installation

```
# Install cert-manager
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "cert-manager" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
cert-manager --version || cert-manager version

# Run initial health check
cert-manager --help

# Configure authentication if required
export CERT_MANAGER_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# cert-manager.yaml or .cert-manager.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: cert-manager-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yml — .github/workflows/cert-manager.yml

```
name: TLS Certificate Management with cert-manager - cert-manager

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Cryptography Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run cert-manager Scan
        run: |
          # Install and configure cert-manager
          echo "Running cert-manager security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: cert-manager-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## cert-manager Security Scan Results\n See workflow artifacts
for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
cert-manager scan \
```

```

--severity CRITICAL,HIGH \
--format json \
--output results.json \
--exit-code 1 \
.

# Parse results programmatically
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:
  .FixedVersion}'

# Generate HTML report
cert-manager scan --format template \
  --template @html.tpl --output report.html .

# Fail pipeline only on CRITICAL
cert-manager scan --exit-code 1 --severity CRITICAL .

```

Step 5: Remediation Workflow

When cert-manager finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```

# Create GitHub issue for each critical finding
cat results.json | jq -r '.Results[].Vulnerabilities[] |
  select(.Severity == "CRITICAL") |
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +
  " | Fix: " + (.FixedVersion // "No fix available")' | \
  while read line; do
    gh issue create --title "$line" --label "security,critical" --body "Auto-
generated from cert-manager scan"
  done

# Track remediation metrics
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length')
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |
  select(.Severity=="CRITICAL")] | length')
echo "Total: $TOTAL | Critical: $CRITICAL"

```

Ultra-Deep Explanation

cert-manager Architecture and Detection Engine

cert-manager uses sophisticated analysis techniques to detect security issues in cryptography contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Cryptography Security Principles

Cryptography security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). cert-manager implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin cert-manager versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating cert-manager's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 24 · LOG MANAGEMENT

Centralized Security Logging with ELK Stack

★ ★ Intermediate · Tools: Elasticsearch, Logstash, Kibana, Filebeat

🎯 Objective

Master Centralized Security Logging with ELK Stack using Elasticsearch, Logstash, Kibana, Filebeat to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

📖 Background & Theory

Log Management is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper log management controls expose themselves to significant security risks, compliance violations, and potential data breaches.

Elasticsearch is a leading tool in the Log Management space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing centralized security logging with elk stack properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

📋 Tasks

Task 1: Install and Configure Elasticsearch

- Install Elasticsearch using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure Elasticsearch to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/elasticsearch-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — Elasticsearch installation

```
# Install Elasticsearch
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "Elasticsearch" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
elasticsearch --version || elasticsearch version

# Run initial health check
elasticsearch --help

# Configure authentication if required
export ELASTICSEARCH_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# elasticsearch.yaml or .elasticsearch.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: elasticsearch-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yaml — .github/workflows/elasticsearch.yml

```
name: Centralized Security Logging with ELK Stack - Elasticsearch

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Log Management Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run Elasticsearch Scan
        run: |
          # Install and configure Elasticsearch
          echo "Running Elasticsearch security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: elasticsearch-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## Elasticsearch Security Scan Results\n See workflow artifacts
for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
elasticsearch scan \
```



```
--severity CRITICAL,HIGH \  
--format json \  
--output results.json \  
--exit-code 1 \  
.  
  
# Parse results programmatically  
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |  
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |  
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:  
  .FixedVersion}'  
  
# Generate HTML report  
elasticsearch scan --format template \  
  --template @html.tpl --output report.html .  
  
# Fail pipeline only on CRITICAL  
elasticsearch scan --exit-code 1 --severity CRITICAL .
```

Step 5: Remediation Workflow

When Elasticsearch finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```
# Create GitHub issue for each critical finding  
cat results.json | jq -r '.Results[].Vulnerabilities[] |  
  select(.Severity == "CRITICAL") |  
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +  
  " | Fix: " + (.FixedVersion // "No fix available")' | \  
while read line; do  
  gh issue create --title "$line" --label "security,critical" --body "Auto-  
generated from Elasticsearch scan"  
done  
  
# Track remediation metrics  
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length']  
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |  
  select(.Severity=="CRITICAL")] | length')  
echo "Total: $TOTAL | Critical: $CRITICAL"
```

Ultra-Deep Explanation

Elasticsearch Architecture and Detection Engine

Elasticsearch uses sophisticated analysis techniques to detect security issues in log management contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false

positive rates. Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Log Management Security Principles

Log Management security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). Elasticsearch implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin Elasticsearch versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating Elasticsearch's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 25 · ACCESS CONTROL**AWS IAM Least Privilege with AWS Access Analyzer**

★ ★ Intermediate · Tools: AWS IAM, Access Analyzer, CloudTrail

Objective

Master AWS IAM Least Privilege with AWS Access Analyzer using AWS IAM, Access Analyzer, CloudTrail to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Access Control is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper access control controls expose themselves to significant security risks, compliance violations, and potential data breaches.

AWS IAM is a leading tool in the Access Control space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing aws iam least privilege with aws access analyzer properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure AWS IAM

- Install AWS IAM using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure AWS IAM to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/aws iam-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — AWS IAM installation

```
# Install AWS IAM
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "AWS IAM" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
aws iam --version || aws iam version

# Run initial health check
aws iam --help

# Configure authentication if required
export AWS_IAM_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# aws iam.yaml or .aws iam.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: aws iam-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yml — .github/workflows/aws iam.yml

```
name: AWS IAM Least Privilege with AWS Access Analyzer - AWS IAM

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Access Control Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run AWS IAM Scan
        run: |
          # Install and configure AWS IAM
          echo "Running AWS IAM security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: aws iam-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## AWS IAM Security Scan Results\n See workflow artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
aws iam scan \
```

```
--severity CRITICAL,HIGH \  
--format json \  
--output results.json \  
--exit-code 1 \  
.  
  
# Parse results programmatically  
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |  
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |  
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:  
  .FixedVersion}'  
  
# Generate HTML report  
aws iam scan --format template \  
  --template @html.tpl --output report.html .  
  
# Fail pipeline only on CRITICAL  
aws iam scan --exit-code 1 --severity CRITICAL .
```

Step 5: Remediation Workflow

When AWS IAM finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```
# Create GitHub issue for each critical finding  
cat results.json | jq -r '.Results[].Vulnerabilities[] |  
  select(.Severity == "CRITICAL") |  
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +  
  " | Fix: " + (.FixedVersion // "No fix available")' | \  
while read line; do  
  gh issue create --title "$line" --label "security,critical" --body "Auto-  
generated from AWS IAM scan"  
done  
  
# Track remediation metrics  
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length ]'  
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |  
  select(.Severity=="CRITICAL")] | length )'  
echo "Total: $TOTAL | Critical: $CRITICAL"
```

Ultra-Deep Explanation

AWS IAM Architecture and Detection Engine

AWS IAM uses sophisticated analysis techniques to detect security issues in access control contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Access Control Security Principles

Access Control security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). AWS IAM implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin AWS IAM versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating AWS IAM's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 26 · GITOPS SECURITY

Secure GitOps with ArgoCD and Vault

★ ★ ★ Advanced · Tools: ArgoCD, Vault, Kubernetes, Sealed Secrets

Objective

Master Secure GitOps with ArgoCD and Vault using ArgoCD, Vault, Kubernetes, Sealed Secrets to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

GitOps Security is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper gitops security controls expose themselves to significant security risks, compliance violations, and potential data breaches.

ArgoCD is a leading tool in the GitOps Security space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing secure gitops with argocd and vault properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure ArgoCD

- Install ArgoCD using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure ArgoCD to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/argocd-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — ArgoCD installation

```
# Install ArgoCD
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "ArgoCD" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
argocd --version || argocd version

# Run initial health check
argocd --help

# Configure authentication if required
export ARGOCD_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# argocd.yaml or .argocd.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: argocd-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yaml — .github/workflows/argocd.yml

```
name: Secure GitOps with ArgoCD and Vault - ArgoCD

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: GitOps Security Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run ArgoCD Scan
        run: |
          # Install and configure ArgoCD
          echo "Running ArgoCD security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: argocd-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## ArgoCD Security Scan Results\n See workflow artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning**bash — Advanced scanning options**

```
# Scan with additional context
argocd scan \
```

```

--severity CRITICAL,HIGH \
--format json \
--output results.json \
--exit-code 1 \
.

# Parse results programmatically
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:
  .FixedVersion}'

# Generate HTML report
argocd scan --format template \
  --template @html.tpl --output report.html .

# Fail pipeline only on CRITICAL
argocd scan --exit-code 1 --severity CRITICAL .

```

Step 5: Remediation Workflow

When ArgoCD finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```

# Create GitHub issue for each critical finding
cat results.json | jq -r '.Results[].Vulnerabilities[] |
  select(.Severity == "CRITICAL") |
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +
  " | Fix: " + (.FixedVersion // "No fix available")' | \
  while read line; do
    gh issue create --title "$line" --label "security,critical" --body "Auto-
generated from ArgoCD scan"
  done

# Track remediation metrics
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length')
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |
  select(.Severity=="CRITICAL")] | length')
echo "Total: $TOTAL | Critical: $CRITICAL"

```

Ultra-Deep Explanation

ArgoCD Architecture and Detection Engine

ArgoCD uses sophisticated analysis techniques to detect security issues in gitops security contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: GitOps Security Security Principles

GitOps Security security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). ArgoCD implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin ArgoCD versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating ArgoCD's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 27 · DAST**API Security Testing with Nuclei**

★ ★ Intermediate · Tools: Nuclei, REST API, OWASP API Top 10

Objective

Master API Security Testing with Nuclei using Nuclei, REST API, OWASP API Top 10 to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

DAST is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper dast controls expose themselves to significant security risks, compliance violations, and potential data breaches.

Nuclei is a leading tool in the DAST space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing api security testing with nuclei properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure Nuclei

- Install Nuclei using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure Nuclei to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/nuclei-scan.yml`

- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — Nuclei installation

```
# Install Nuclei
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "Nuclei" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
nuclei --version || nuclei version

# Run initial health check
nuclei --help

# Configure authentication if required
export NUCLEI_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# nuclei.yaml or .nuclei.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: nuclei-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yaml — .github/workflows/nuclei.yml

```
name: API Security Testing with Nuclei - Nuclei

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: DAST Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run Nuclei Scan
        run: |
          # Install and configure Nuclei
          echo "Running Nuclei security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: nuclei-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## Nuclei Security Scan Results\n See workflow artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
nuclei scan \
  --severity CRITICAL,HIGH \
```

```

--format json \
--output results.json \
--exit-code 1 \
.

# Parse results programmatically
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:
  .FixedVersion}'

# Generate HTML report
nuclei scan --format template \
  --template @html.tpl --output report.html .

# Fail pipeline only on CRITICAL
nuclei scan --exit-code 1 --severity CRITICAL .

```

Step 5: Remediation Workflow

When Nuclei finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```

# Create GitHub issue for each critical finding
cat results.json | jq -r '.Results[].Vulnerabilities[] |
  select(.Severity == "CRITICAL") |
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +
  " | Fix: " + (.FixedVersion // "No fix available")' | \
  while read line; do
    gh issue create --title "$line" --label "security,critical" --body "Auto-
generated from Nuclei scan"
  done

# Track remediation metrics
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length']
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |
  select(.Severity=="CRITICAL")] | length')
echo "Total: $TOTAL | Critical: $CRITICAL"

```

Ultra-Deep Explanation

Nuclei Architecture and Detection Engine

Nuclei uses sophisticated analysis techniques to detect security issues in dast contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates. Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: DAST Security Principles

DAST security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). Nuclei implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin Nuclei versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating Nuclei's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 28 · CONTAINER SECURITY**Container Hardening with Distroless and Rootless Images**

★ ★ Intermediate · Tools: Docker, Distroless, Buildah, Podman

Objective

Master Container Hardening with Distroless and Rootless Images using Docker, Distroless, Buildah, Podman to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Container Security is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper container security controls expose themselves to significant security risks, compliance violations, and potential data breaches.

Docker is a leading tool in the Container Security space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing container hardening with distroless and rootless images properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure Docker

- Install Docker using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure Docker to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/docker-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — Docker installation

```
# Install Docker
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "Docker" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
docker --version || docker version

# Run initial health check
docker --help

# Configure authentication if required
export DOCKER_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# docker.yaml or .docker.yml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: docker-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yml — .github/workflows/docker.yml

```
name: Container Hardening with Distroless and Rootless Images - Docker

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Container Security Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run Docker Scan
        run: |
          # Install and configure Docker
          echo "Running Docker security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: docker-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## Docker Security Scan Results\n See workflow artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
docker scan \
```

```
--severity CRITICAL,HIGH \  
--format json \  
--output results.json \  
--exit-code 1 \  
.  
  
# Parse results programmatically  
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |  
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |  
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:  
  .FixedVersion}'  
  
# Generate HTML report  
docker scan --format template \  
  --template @html.tpl --output report.html .  
  
# Fail pipeline only on CRITICAL  
docker scan --exit-code 1 --severity CRITICAL .
```

Step 5: Remediation Workflow

When Docker finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```
# Create GitHub issue for each critical finding  
cat results.json | jq -r '.Results[].Vulnerabilities[] |  
  select(.Severity == "CRITICAL") |  
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +  
  " | Fix: " + (.FixedVersion // "No fix available")' | \  
while read line; do  
  gh issue create --title "$line" --label "security,critical" --body "Auto-  
generated from Docker scan"  
done  
  
# Track remediation metrics  
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length ]'  
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |  
  select(.Severity=="CRITICAL")] | length )'  
echo "Total: $TOTAL | Critical: $CRITICAL"
```

Ultra-Deep Explanation

Docker Architecture and Detection Engine

Docker uses sophisticated analysis techniques to detect security issues in container security contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Container Security Security Principles

Container Security security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). Docker implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin Docker versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating Docker's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 29 · IAC SECURITY**Kubernetes Manifest Security Scanning with Kubesec**

★ ★ Intermediate · Tools: Kubesec, Datree, Polaris, Kubernetes

Objective

Master Kubernetes Manifest Security Scanning with Kubesec using Kubesec, Datree, Polaris, Kubernetes to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

IaC Security is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper iac security controls expose themselves to significant security risks, compliance violations, and potential data breaches.

Kubesec is a leading tool in the IaC Security space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing kubernetes manifest security scanning with kubesec properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure Kubesec

- Install Kubesec using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure Kubesec to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/kubeseccan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — Kubesecc installation

```
# Install Kubesecc
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "Kubesecc" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
kubesecc --version || kubesecc version

# Run initial health check
kubesecc --help

# Configure authentication if required
export KUBESEC_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# kubesecc.yaml or .kubesecc.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: kubesecc-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yaml — .github/workflows/kubesecc.yml

```
name: Kubernetes Manifest Security Scanning with Kubesec - Kubesec

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: IaC Security Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run Kubesec Scan
        run: |
          # Install and configure Kubesec
          echo "Running Kubesec security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: kubesecc-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## Kubesec Security Scan Results\n See workflow artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
kubesec scan \
```

```

--severity CRITICAL,HIGH \
--format json \
--output results.json \
--exit-code 1 \
.

# Parse results programmatically
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:
  .FixedVersion}'

# Generate HTML report
kubesec scan --format template \
  --template @html.tpl --output report.html .

# Fail pipeline only on CRITICAL
kubesec scan --exit-code 1 --severity CRITICAL .

```

Step 5: Remediation Workflow

When Kubesec finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```

# Create GitHub issue for each critical finding
cat results.json | jq -r '.Results[].Vulnerabilities[] |
  select(.Severity == "CRITICAL") |
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +
  " | Fix: " + (.FixedVersion // "No fix available")' | \
while read line; do
  gh issue create --title "$line" --label "security,critical" --body "Auto-
generated from Kubesec scan"
done

# Track remediation metrics
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length')
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |
  select(.Severity=="CRITICAL")] | length')
echo "Total: $TOTAL | Critical: $CRITICAL"

```

Ultra-Deep Explanation

Kubesec Architecture and Detection Engine

Kubesec uses sophisticated analysis techniques to detect security issues in iac security contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: IaC Security Principles

IaC Security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). Kubesec implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin Kubesec versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating Kubesec's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 30 · INCIDENT RESPONSE

Security Incident Response Runbook

★ ★ Intermediate · Tools: PagerDuty, Slack, Jira, Runbook

Objective

Master Security Incident Response Runbook using PagerDuty, Slack, Jira, Runbook to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Incident Response is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper incident response controls expose themselves to significant security risks, compliance violations, and potential data breaches.

PagerDuty is a leading tool in the Incident Response space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing security incident response runbook properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure PagerDuty

- Install PagerDuty using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure PagerDuty to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/pagerduty-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — PagerDuty installation

```
# Install PagerDuty
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "PagerDuty" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
pagerduty --version || pagerduty version

# Run initial health check
pagerduty --help

# Configure authentication if required
export PAGERDUTY_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# pagerduty.yaml or .pagerduty.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: pagerduty-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yml — .github/workflows/pagerduty.yml

```
name: Security Incident Response Runbook - PagerDuty

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Incident Response Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run PagerDuty Scan
        run: |
          # Install and configure PagerDuty
          echo "Running PagerDuty security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: pagerduty-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## PagerDuty Security Scan Results\n See workflow artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
pagerduty scan \
```

```
--severity CRITICAL,HIGH \  
--format json \  
--output results.json \  
--exit-code 1 \  
.  
  
# Parse results programmatically  
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |  
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |  
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:  
  .FixedVersion}'  
  
# Generate HTML report  
pagerduty scan --format template \  
  --template @html.tpl --output report.html .  
  
# Fail pipeline only on CRITICAL  
pagerduty scan --exit-code 1 --severity CRITICAL .
```

Step 5: Remediation Workflow

When PagerDuty finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```
# Create GitHub issue for each critical finding  
cat results.json | jq -r '.Results[].Vulnerabilities[] |  
  select(.Severity == "CRITICAL") |  
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +  
  " | Fix: " + (.FixedVersion // "No fix available")' | \  
while read line; do  
  gh issue create --title "$line" --label "security,critical" --body "Auto-  
generated from PagerDuty scan"  
done  
  
# Track remediation metrics  
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length ]'  
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |  
  select(.Severity=="CRITICAL")] | length )'  
echo "Total: $TOTAL | Critical: $CRITICAL"
```

Ultra-Deep Explanation

PagerDuty Architecture and Detection Engine

PagerDuty uses sophisticated analysis techniques to detect security issues in incident response contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Incident Response Security Principles

Incident Response security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). PagerDuty implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin PagerDuty versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating PagerDuty's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 31 · CLOUD SECURITY**AWS GuardDuty Threat Detection**

★ ★ Intermediate · Tools: AWS GuardDuty, CloudWatch, Lambda

Objective

Master AWS GuardDuty Threat Detection using AWS GuardDuty, CloudWatch, Lambda to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Cloud Security is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper cloud security controls expose themselves to significant security risks, compliance violations, and potential data breaches.

AWS GuardDuty is a leading tool in the Cloud Security space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing aws guarddduty threat detection properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure AWS GuardDuty

- Install AWS GuardDuty using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure AWS GuardDuty to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/aws-guardduty-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — AWS GuardDuty installation

```
# Install AWS GuardDuty
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "AWS GuardDuty" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
aws guardduty --version || aws guardduty version

# Run initial health check
aws guardduty --help

# Configure authentication if required
export AWS_GUARDDUTY_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# aws guardduty.yaml or .aws guardduty.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: aws-guardduty-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yml — .github/workflows/aws_guarddduty.yml

```
name: AWS GuardDuty Threat Detection - AWS GuardDuty

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Cloud Security Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run AWS GuardDuty Scan
        run: |
          # Install and configure AWS GuardDuty
          echo "Running AWS GuardDuty security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: aws_guarddduty-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## AWS GuardDuty Security Scan Results\n See workflow artifacts
for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
aws guarddduty scan \
```

```
--severity CRITICAL,HIGH \  
--format json \  
--output results.json \  
--exit-code 1 \  
.  
  
# Parse results programmatically  
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |  
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |  
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:  
  .FixedVersion}'  
  
# Generate HTML report  
aws guardduty scan --format template \  
  --template @html.tpl --output report.html .  
  
# Fail pipeline only on CRITICAL  
aws guardduty scan --exit-code 1 --severity CRITICAL .
```

Step 5: Remediation Workflow

When AWS GuardDuty finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```
# Create GitHub issue for each critical finding  
cat results.json | jq -r '.Results[].Vulnerabilities[] |  
  select(.Severity == "CRITICAL") |  
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +  
  " | Fix: " + (.FixedVersion // "No fix available")' | \  
while read line; do  
  gh issue create --title "$line" --label "security,critical" --body "Auto-  
generated from AWS GuardDuty scan"  
done  
  
# Track remediation metrics  
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length']  
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |  
  select(.Severity=="CRITICAL")] | length')  
echo "Total: $TOTAL | Critical: $CRITICAL"
```

Ultra-Deep Explanation

AWS GuardDuty Architecture and Detection Engine

AWS GuardDuty uses sophisticated analysis techniques to detect security issues in cloud security contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false

positive rates. Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Cloud Security Security Principles

Cloud Security security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). AWS GuardDuty implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin AWS GuardDuty versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating AWS GuardDuty's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 32 · COMPLIANCE

SOC 2 Controls Implementation for DevOps

☆☆☆ Advanced · Tools: AWS Config, CloudTrail, SecurityHub

Objective

Master SOC 2 Controls Implementation for DevOps using AWS Config, CloudTrail, SecurityHub to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Compliance is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper compliance controls expose themselves to significant security risks, compliance violations, and potential data breaches.

AWS Config is a leading tool in the Compliance space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing soc 2 controls implementation for devops properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure AWS Config

- Install AWS Config using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure AWS Config to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create .github/workflows/aws config-scan.yml
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — AWS Config installation

```
# Install AWS Config
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "AWS Config" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
aws config --version || aws config version

# Run initial health check
aws config --help

# Configure authentication if required
export AWS_CONFIG_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# aws config.yaml or .aws config.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: aws config-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yml — .github/workflows/aws config.yml

```
name: SOC 2 Controls Implementation for DevOps - AWS Config

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Compliance Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run AWS Config Scan
        run: |
          # Install and configure AWS Config
          echo "Running AWS Config security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: aws config-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## AWS Config Security Scan Results\n See workflow artifacts
for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
aws config scan \
```

```
--severity CRITICAL,HIGH \
--format json \
--output results.json \
--exit-code 1 \
.

# Parse results programmatically
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:
  .FixedVersion}'

# Generate HTML report
aws config scan --format template \
  --template @html.tpl --output report.html .

# Fail pipeline only on CRITICAL
aws config scan --exit-code 1 --severity CRITICAL .
```

Step 5: Remediation Workflow

When AWS Config finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```
# Create GitHub issue for each critical finding
cat results.json | jq -r '.Results[].Vulnerabilities[] |
  select(.Severity == "CRITICAL") |
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +
  " | Fix: " + (.FixedVersion // "No fix available")' | \
while read line; do
  gh issue create --title "$line" --label "security,critical" --body "Auto-
generated from AWS Config scan"
done

# Track remediation metrics
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length')
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |
  select(.Severity=="CRITICAL")] | length')
echo "Total: $TOTAL | Critical: $CRITICAL"
```

Ultra-Deep Explanation

AWS Config Architecture and Detection Engine

AWS Config uses sophisticated analysis techniques to detect security issues in compliance contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Compliance Security Principles

Compliance security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). AWS Config implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin AWS Config versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating AWS Config's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 33 · CONTAINER SECURITY**Multi-stage Dockerfile Security Best Practices**

★ Beginner · Tools: Docker, Hadolint, Checkov

Objective

Master Multi-stage Dockerfile Security Best Practices using Docker, Hadolint, Checkov to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Container Security is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper container security controls expose themselves to significant security risks, compliance violations, and potential data breaches.

Docker is a leading tool in the Container Security space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing multi-stage dockerfile security best practices properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure Docker

- Install Docker using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure Docker to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/docker-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — Docker installation

```
# Install Docker
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "Docker" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
docker --version || docker version

# Run initial health check
docker --help

# Configure authentication if required
export DOCKER_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# docker.yaml or .docker.yml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: docker-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yml — .github/workflows/docker.yml

```
name: Multi-stage Dockerfile Security Best Practices - Docker

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Container Security Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run Docker Scan
        run: |
          # Install and configure Docker
          echo "Running Docker security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: docker-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## Docker Security Scan Results\n See workflow artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
docker scan \
```

```

--severity CRITICAL,HIGH \
--format json \
--output results.json \
--exit-code 1 \
.

# Parse results programmatically
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:
  .FixedVersion}'

# Generate HTML report
docker scan --format template \
  --template @html.tpl --output report.html .

# Fail pipeline only on CRITICAL
docker scan --exit-code 1 --severity CRITICAL .

```

Step 5: Remediation Workflow

When Docker finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```

# Create GitHub issue for each critical finding
cat results.json | jq -r '.Results[].Vulnerabilities[] |
  select(.Severity == "CRITICAL") |
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +
  " | Fix: " + (.FixedVersion // "No fix available")' | \
while read line; do
  gh issue create --title "$line" --label "security,critical" --body "Auto-
generated from Docker scan"
done

# Track remediation metrics
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length')
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |
select(.Severity=="CRITICAL")] | length')
echo "Total: $TOTAL | Critical: $CRITICAL"

```

Ultra-Deep Explanation

Docker Architecture and Detection Engine

Docker uses sophisticated analysis techniques to detect security issues in container security contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Container Security Security Principles

Container Security security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). Docker implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin Docker versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating Docker's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 34 · SECRETS MANAGEMENT

AWS Secrets Manager Auto-Rotation

★ ★ Intermediate · Tools: AWS Secrets Manager, Lambda, RDS

Objective

Master AWS Secrets Manager Auto-Rotation using AWS Secrets Manager, Lambda, RDS to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Secrets Management is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper secrets management controls expose themselves to significant security risks, compliance violations, and potential data breaches.

AWS Secrets Manager is a leading tool in the Secrets Management space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing aws secrets manager auto-rotation properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure AWS Secrets Manager

- Install AWS Secrets Manager using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure AWS Secrets Manager to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create .github/workflows/aws secrets manager-scan.yml
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — AWS Secrets Manager installation

```
# Install AWS Secrets Manager
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "AWS Secrets Manager" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
aws secrets manager --version || aws secrets manager version

# Run initial health check
aws secrets manager --help

# Configure authentication if required
export AWS_SECRETS_MANAGER_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# aws secrets manager.yaml or .aws secrets manager.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: aws secrets manager-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yml — .github/workflows/aws secrets manager.yml

```
name: AWS Secrets Manager Auto-Rotation - AWS Secrets Manager

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Secrets Management Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run AWS Secrets Manager Scan
        run: |
          # Install and configure AWS Secrets Manager
          echo "Running AWS Secrets Manager security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: aws secrets manager-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## AWS Secrets Manager Security Scan Results\n See workflow
artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
aws secrets manager scan \
```

```

--severity CRITICAL,HIGH \
--format json \
--output results.json \
--exit-code 1 \
.

# Parse results programmatically
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:
  .FixedVersion}'

# Generate HTML report
aws secrets manager scan --format template \
  --template @html.tpl --output report.html .

# Fail pipeline only on CRITICAL
aws secrets manager scan --exit-code 1 --severity CRITICAL .

```

Step 5: Remediation Workflow

When AWS Secrets Manager finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```

# Create GitHub issue for each critical finding
cat results.json | jq -r '.Results[].Vulnerabilities[] |
  select(.Severity == "CRITICAL") |
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +
  " | Fix: " + (.FixedVersion // "No fix available")' | \
  while read line; do
    gh issue create --title "$line" --label "security,critical" --body "Auto-
generated from AWS Secrets Manager scan"
  done

# Track remediation metrics
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length')
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |
  select(.Severity=="CRITICAL")] | length')
echo "Total: $TOTAL | Critical: $CRITICAL"

```

Ultra-Deep Explanation

AWS Secrets Manager Architecture and Detection Engine

AWS Secrets Manager uses sophisticated analysis techniques to detect security issues in secrets management contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while

managing false positive rates. Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Secrets Management Security Principles

Secrets Management security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). AWS Secrets Manager implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin AWS Secrets Manager versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating AWS Secrets Manager's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 35 · NETWORK SECURITY

Web Application Firewall with AWS WAF

☆☆ Intermediate · Tools: AWS WAF, CloudFront, ALB

Objective

Master Web Application Firewall with AWS WAF using AWS WAF, CloudFront, ALB to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Network Security is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper network security controls expose themselves to significant security risks, compliance violations, and potential data breaches.

AWS WAF is a leading tool in the Network Security space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing web application firewall with aws waf properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure AWS WAF

- Install AWS WAF using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure AWS WAF to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/aws waf-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — AWS WAF installation

```
# Install AWS WAF
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "AWS WAF" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
aws waf --version || aws waf version

# Run initial health check
aws waf --help

# Configure authentication if required
export AWS_WAF_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# aws waf.yaml or .aws waf.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: aws waf-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yml — .github/workflows/aws waf.yml

```
name: Web Application Firewall with AWS WAF - AWS WAF

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Network Security Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run AWS WAF Scan
        run: |
          # Install and configure AWS WAF
          echo "Running AWS WAF security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: aws waf-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## AWS WAF Security Scan Results\n See workflow artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning**bash — Advanced scanning options**

```
# Scan with additional context
aws waf scan \
```

```
--severity CRITICAL,HIGH \  
--format json \  
--output results.json \  
--exit-code 1 \  
.  
  
# Parse results programmatically  
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |  
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |  
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:  
  .FixedVersion}'  
  
# Generate HTML report  
aws waf scan --format template \  
  --template @html.tpl --output report.html .  
  
# Fail pipeline only on CRITICAL  
aws waf scan --exit-code 1 --severity CRITICAL .
```

Step 5: Remediation Workflow

When AWS WAF finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```
# Create GitHub issue for each critical finding  
cat results.json | jq -r '.Results[].Vulnerabilities[] |  
  select(.Severity == "CRITICAL") |  
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +  
  " | Fix: " + (.FixedVersion // "No fix available")' | \  
while read line; do  
  gh issue create --title "$line" --label "security,critical" --body "Auto-  
generated from AWS WAF scan"  
done  
  
# Track remediation metrics  
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length ]'  
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |  
  select(.Severity=="CRITICAL")] | length )'  
echo "Total: $TOTAL | Critical: $CRITICAL"
```

Ultra-Deep Explanation

AWS WAF Architecture and Detection Engine

AWS WAF uses sophisticated analysis techniques to detect security issues in network security contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Network Security Security Principles

Network Security security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). AWS WAF implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin AWS WAF versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating AWS WAF's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 36 · CI/CD SECURITY

Dependency Vulnerability Scanning with Snyk

★ Beginner · Tools: Snyk, npm, pip, GitHub Actions

Objective

Master Dependency Vulnerability Scanning with Snyk using Snyk, npm, pip, GitHub Actions to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

CI/CD Security is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper ci/cd security controls expose themselves to significant security risks, compliance violations, and potential data breaches.

Snyk is a leading tool in the CI/CD Security space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing dependency vulnerability scanning with snyk properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure Snyk

- Install Snyk using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure Snyk to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/snyk-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — Snyk installation

```
# Install Snyk
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "Snyk" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
snyk --version || snyk version

# Run initial health check
snyk --help

# Configure authentication if required
export SNYK_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# snyk.yaml or .snyk.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: snyk-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yml — .github/workflows/snyk.yml

```
name: Dependency Vulnerability Scanning with Snyk - Snyk

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: CI/CD Security Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run Snyk Scan
        run: |
          # Install and configure Snyk
          echo "Running Snyk security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: snyk-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## Snyk Security Scan Results\n See workflow artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning**bash — Advanced scanning options**

```
# Scan with additional context
snyk scan \
```

```

--severity CRITICAL,HIGH \
--format json \
--output results.json \
--exit-code 1 \
.

# Parse results programmatically
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:
  .FixedVersion}'

# Generate HTML report
snyc scan --format template \
  --template @html.tpl --output report.html .

# Fail pipeline only on CRITICAL
snyc scan --exit-code 1 --severity CRITICAL .

```

Step 5: Remediation Workflow

When Snyk finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```

# Create GitHub issue for each critical finding
cat results.json | jq -r '.Results[].Vulnerabilities[] |
  select(.Severity == "CRITICAL") |
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +
  " | Fix: " + (.FixedVersion // "No fix available")' | \
while read line; do
  gh issue create --title "$line" --label "security,critical" --body "Auto-
generated from Snyk scan"
done

# Track remediation metrics
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length')
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |
select(.Severity=="CRITICAL")] | length')
echo "Total: $TOTAL | Critical: $CRITICAL"

```

Ultra-Deep Explanation

Snyk Architecture and Detection Engine

Snyk uses sophisticated analysis techniques to detect security issues in ci/cd security contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: CI/CD Security Security Principles

CI/CD Security security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). Snyk implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin Snyk versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating Snyk's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 37 · KUBERNETES SECURITY

Kubernetes Audit Logging Deep Dive

★ ★ ★ Advanced · Tools: Kubernetes, Audit Policy, Fluentd

Objective

Master Kubernetes Audit Logging Deep Dive using Kubernetes, Audit Policy, Fluentd to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Kubernetes Security is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper kubernetes security controls expose themselves to significant security risks, compliance violations, and potential data breaches.

Kubernetes is a leading tool in the Kubernetes Security space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing kubernetes audit logging deep dive properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure Kubernetes

- Install Kubernetes using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure Kubernetes to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/kubernetes-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

✓ Complete Solution

Step 1: Installation and Initial Setup

bash — Kubernetes installation

```
# Install Kubernetes
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "Kubernetes" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
kubernetes --version || kubernetes version

# Run initial health check
kubernetes --help

# Configure authentication if required
export KUBERNETES_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# kubernetes.yaml or .kubernetes.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: kubernetes-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yaml — .github/workflows/kubernetes.yml

```
name: Kubernetes Audit Logging Deep Dive - Kubernetes

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Kubernetes Security Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run Kubernetes Scan
        run: |
          # Install and configure Kubernetes
          echo "Running Kubernetes security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: kubernetes-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## Kubernetes Security Scan Results\n See workflow artifacts
for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
kubernetes scan \
```

```
--severity CRITICAL,HIGH \  
--format json \  
--output results.json \  
--exit-code 1 \  
.  
  
# Parse results programmatically  
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |  
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |  
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:  
  .FixedVersion}'  
  
# Generate HTML report  
kubernetes scan --format template \  
  --template @html.tpl --output report.html .  
  
# Fail pipeline only on CRITICAL  
kubernetes scan --exit-code 1 --severity CRITICAL .
```

Step 5: Remediation Workflow

When Kubernetes finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```
# Create GitHub issue for each critical finding  
cat results.json | jq -r '.Results[].Vulnerabilities[] |  
  select(.Severity == "CRITICAL") |  
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +  
  " | Fix: " + (.FixedVersion // "No fix available")' | \  
while read line; do  
  gh issue create --title "$line" --label "security,critical" --body "Auto-  
generated from Kubernetes scan"  
done  
  
# Track remediation metrics  
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length'] )  
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |  
  select(.Severity=="CRITICAL")] | length')  
echo "Total: $TOTAL | Critical: $CRITICAL"
```

Ultra-Deep Explanation

Kubernetes Architecture and Detection Engine

Kubernetes uses sophisticated analysis techniques to detect security issues in kubernetes security contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false

positive rates. Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Kubernetes Security Principles

Kubernetes Security security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). Kubernetes implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin Kubernetes versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating Kubernetes's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 38 · IDENTITY & ACCESS

OIDC Federated Identity for GitHub Actions to AWS

★ ★ ★ Advanced · Tools: GitHub Actions, AWS OIDC, IAM Roles

Objective

Master OIDC Federated Identity for GitHub Actions to AWS using GitHub Actions, AWS OIDC, IAM Roles to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Identity & Access is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper identity & access controls expose themselves to significant security risks, compliance violations, and potential data breaches.

GitHub Actions is a leading tool in the Identity & Access space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing oidc federated identity for github actions to aws properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure GitHub Actions

- Install GitHub Actions using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure GitHub Actions to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/github actions-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — GitHub Actions installation

```
# Install GitHub Actions
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "GitHub Actions" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
github actions --version || github actions version

# Run initial health check
github actions --help

# Configure authentication if required
export GITHUB_ACTIONS_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# github actions.yaml or .github actions.yml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: github actions-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yaml — .github/workflows/github actions.yml

```
name: OIDC Federated Identity for GitHub Actions to AWS - GitHub Actions

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Identity & Access Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run GitHub Actions Scan
        run: |
          # Install and configure GitHub Actions
          echo "Running GitHub Actions security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: github actions-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## GitHub Actions Security Scan Results\n See workflow
artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning**bash — Advanced scanning options**

```
# Scan with additional context
github actions scan \
```

```
--severity CRITICAL,HIGH \  
--format json \  
--output results.json \  
--exit-code 1 \  
.  
  
# Parse results programmatically  
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |  
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |  
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:  
  .FixedVersion}'  
  
# Generate HTML report  
github actions scan --format template \  
  --template @html.tpl --output report.html .  
  
# Fail pipeline only on CRITICAL  
github actions scan --exit-code 1 --severity CRITICAL .
```

Step 5: Remediation Workflow

When GitHub Actions finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```
# Create GitHub issue for each critical finding  
cat results.json | jq -r '.Results[].Vulnerabilities[] |  
  select(.Severity == "CRITICAL") |  
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +  
  " | Fix: " + (.FixedVersion // "No fix available")' | \  
while read line; do  
  gh issue create --title "$line" --label "security,critical" --body "Auto-  
generated from GitHub Actions scan"  
done  
  
# Track remediation metrics  
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length']  
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |  
  select(.Severity=="CRITICAL")] | length')  
echo "Total: $TOTAL | Critical: $CRITICAL"
```

Ultra-Deep Explanation

GitHub Actions Architecture and Detection Engine

GitHub Actions uses sophisticated analysis techniques to detect security issues in identity & access contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false

positive rates. Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Identity & Access Security Principles

Identity & Access security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). GitHub Actions implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin GitHub Actions versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating GitHub Actions's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 39 · COMPLIANCE

DORA Metrics for DevSecOps Teams

☆☆ Intermediate · Tools: GitHub, Datadog, Four Keys Framework

Objective

Master DORA Metrics for DevSecOps Teams using GitHub, Datadog, Four Keys Framework to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Compliance is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper compliance controls expose themselves to significant security risks, compliance violations, and potential data breaches.

GitHub is a leading tool in the Compliance space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing dora metrics for devsecops teams properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure GitHub

- Install GitHub using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure GitHub to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/github-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

✓ Complete Solution

Step 1: Installation and Initial Setup

bash — GitHub installation

```
# Install GitHub
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "GitHub" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
github --version || github version

# Run initial health check
github --help

# Configure authentication if required
export GITHUB_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# github.yaml or .github.yml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: github-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yaml — .github/workflows/github.yml

```
name: DORA Metrics for DevSecOps Teams - GitHub

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Compliance Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run GitHub Scan
        run: |
          # Install and configure GitHub
          echo "Running GitHub security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: github-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## GitHub Security Scan Results\n See workflow artifacts for
details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
github scan \
```

```
--severity CRITICAL,HIGH \  
--format json \  
--output results.json \  
--exit-code 1 \  
.  
  
# Parse results programmatically  
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |  
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |  
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:  
  .FixedVersion}'  
  
# Generate HTML report  
github scan --format template \  
  --template @html.tpl --output report.html .  
  
# Fail pipeline only on CRITICAL  
github scan --exit-code 1 --severity CRITICAL .
```

Step 5: Remediation Workflow

When GitHub finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```
# Create GitHub issue for each critical finding  
cat results.json | jq -r '.Results[].Vulnerabilities[] |  
  select(.Severity == "CRITICAL") |  
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +  
  " | Fix: " + (.FixedVersion // "No fix available")' | \  
while read line; do  
  gh issue create --title "$line" --label "security,critical" --body "Auto-  
generated from GitHub scan"  
done  
  
# Track remediation metrics  
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length ]'  
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |  
select(.Severity=="CRITICAL")] | length )'  
echo "Total: $TOTAL | Critical: $CRITICAL"
```

Ultra-Deep Explanation

GitHub Architecture and Detection Engine

GitHub uses sophisticated analysis techniques to detect security issues in compliance contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Compliance Security Principles

Compliance security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). GitHub implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin GitHub versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating GitHub's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 40 · SCANNING

License Compliance Scanning with FOSSA

★ Beginner · Tools: FOSSA, GitHub Actions, CycloneDX

Objective

Master License Compliance Scanning with FOSSA using FOSSA, GitHub Actions, CycloneDX to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Scanning is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper scanning controls expose themselves to significant security risks, compliance violations, and potential data breaches.

FOSSA is a leading tool in the Scanning space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing license compliance scanning with fossa properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure FOSSA

- Install FOSSA using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure FOSSA to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/fossa-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — FOSSA installation

```
# Install FOSSA
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "FOSSA" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
fossa --version || fossa version

# Run initial health check
fossa --help

# Configure authentication if required
export FOSSA_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# fossa.yaml or .fossa.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: fossa-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yaml — .github/workflows/fossa.yml

```
name: License Compliance Scanning with FOSSA - FOSSA

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Scanning Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run FOSSA Scan
        run: |
          # Install and configure FOSSA
          echo "Running FOSSA security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: fossa-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## FOSSA Security Scan Results\n See workflow artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
fossa scan \
```

```
--severity CRITICAL,HIGH \  
--format json \  
--output results.json \  
--exit-code 1 \  
.  
  
# Parse results programmatically  
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |  
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |  
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:  
  .FixedVersion}'  
  
# Generate HTML report  
fossa scan --format template \  
  --template @html.tpl --output report.html .  
  
# Fail pipeline only on CRITICAL  
fossa scan --exit-code 1 --severity CRITICAL .
```

Step 5: Remediation Workflow

When FOSSA finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```
# Create GitHub issue for each critical finding  
cat results.json | jq -r '.Results[].Vulnerabilities[] |  
  select(.Severity == "CRITICAL") |  
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +  
  " | Fix: " + (.FixedVersion // "No fix available")' | \  
while read line; do  
  gh issue create --title "$line" --label "security,critical" --body "Auto-  
generated from FOSSA scan"  
done  
  
# Track remediation metrics  
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length']  
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |  
  select(.Severity=="CRITICAL")] | length')  
echo "Total: $TOTAL | Critical: $CRITICAL"
```

Ultra-Deep Explanation

FOSSA Architecture and Detection Engine

FOSSA uses sophisticated analysis techniques to detect security issues in scanning contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Scanning Security Principles

Scanning security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). FOSSA implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin FOSSA versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating FOSSA's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 41 · RUNTIME SECURITY

Kubernetes Admission Controllers for Security

★ ★ ★ Advanced · Tools: Kubernetes, Kyverno, OPA, Webhook

Objective

Master Kubernetes Admission Controllers for Security using Kubernetes, Kyverno, OPA, Webhook to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Runtime Security is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper runtime security controls expose themselves to significant security risks, compliance violations, and potential data breaches.

Kubernetes is a leading tool in the Runtime Security space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing kubernetes admission controllers for security properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure Kubernetes

- Install Kubernetes using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure Kubernetes to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/kubernetes-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — Kubernetes installation

```
# Install Kubernetes
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "Kubernetes" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
kubernetes --version || kubernetes version

# Run initial health check
kubernetes --help

# Configure authentication if required
export KUBERNETES_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# kubernetes.yaml or .kubernetes.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: kubernetes-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yaml — .github/workflows/kubernetes.yml

```
name: Kubernetes Admission Controllers for Security - Kubernetes

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Runtime Security Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run Kubernetes Scan
        run: |
          # Install and configure Kubernetes
          echo "Running Kubernetes security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: kubernetes-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## Kubernetes Security Scan Results\n See workflow artifacts
for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
kubernetes scan \
```

```
--severity CRITICAL,HIGH \
--format json \
--output results.json \
--exit-code 1 \
.

# Parse results programmatically
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:
  .FixedVersion}'

# Generate HTML report
kubernetes scan --format template \
  --template @html.tpl --output report.html .

# Fail pipeline only on CRITICAL
kubernetes scan --exit-code 1 --severity CRITICAL .
```

Step 5: Remediation Workflow

When Kubernetes finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```
# Create GitHub issue for each critical finding
cat results.json | jq -r '.Results[].Vulnerabilities[] |
  select(.Severity == "CRITICAL") |
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +
  " | Fix: " + (.FixedVersion // "No fix available")' | \
  while read line; do
    gh issue create --title "$line" --label "security,critical" --body "Auto-
generated from Kubernetes scan"
  done

# Track remediation metrics
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length')
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |
  select(.Severity=="CRITICAL")] | length')
echo "Total: $TOTAL | Critical: $CRITICAL"
```

Ultra-Deep Explanation

Kubernetes Architecture and Detection Engine

Kubernetes uses sophisticated analysis techniques to detect security issues in runtime security contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Runtime Security Security Principles

Runtime Security security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). Kubernetes implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin Kubernetes versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating Kubernetes's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 42 · ENCRYPTION**Data Encryption at Rest and in Transit**

★ ★ Intermediate · Tools: AWS KMS, Kubernetes Secrets Encryption

Objective

Master Data Encryption at Rest and in Transit using AWS KMS, Kubernetes Secrets Encryption to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Encryption is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper encryption controls expose themselves to significant security risks, compliance violations, and potential data breaches.

AWS KMS is a leading tool in the Encryption space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing data encryption at rest and in transit properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure AWS KMS

- Install AWS KMS using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure AWS KMS to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/aws kms-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — AWS KMS installation

```
# Install AWS KMS
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "AWS KMS" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
aws kms --version || aws kms version

# Run initial health check
aws kms --help

# Configure authentication if required
export AWS_KMS_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# aws kms.yaml or .aws kms.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: aws kms-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yml — .github/workflows/aws kms.yml

```
name: Data Encryption at Rest and in Transit - AWS KMS

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Encryption Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run AWS KMS Scan
        run: |
          # Install and configure AWS KMS
          echo "Running AWS KMS security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: aws kms-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## AWS KMS Security Scan Results\n See workflow artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
aws kms scan \
```

```
--severity CRITICAL,HIGH \  
--format json \  
--output results.json \  
--exit-code 1 \  
.  
  
# Parse results programmatically  
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |  
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |  
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:  
  .FixedVersion}'  
  
# Generate HTML report  
aws kms scan --format template \  
  --template @html.tpl --output report.html .  
  
# Fail pipeline only on CRITICAL  
aws kms scan --exit-code 1 --severity CRITICAL .
```

Step 5: Remediation Workflow

When AWS KMS finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```
# Create GitHub issue for each critical finding  
cat results.json | jq -r '.Results[].Vulnerabilities[] |  
  select(.Severity == "CRITICAL") |  
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +  
  " | Fix: " + (.FixedVersion // "No fix available")' | \  
while read line; do  
  gh issue create --title "$line" --label "security,critical" --body "Auto-  
generated from AWS KMS scan"  
done  
  
# Track remediation metrics  
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length ]'  
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |  
  select(.Severity=="CRITICAL")] | length )'  
echo "Total: $TOTAL | Critical: $CRITICAL"
```

Ultra-Deep Explanation

AWS KMS Architecture and Detection Engine

AWS KMS uses sophisticated analysis techniques to detect security issues in encryption contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Encryption Security Principles

Encryption security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). AWS KMS implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin AWS KMS versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating AWS KMS's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 43 · CLOUD SECURITY**S3 Bucket Security Audit and Remediation**

★ Beginner · Tools: AWS S3, Terraform, Checkov, Config

Objective

Master S3 Bucket Security Audit and Remediation using AWS S3, Terraform, Checkov, Config to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Cloud Security is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper cloud security controls expose themselves to significant security risks, compliance violations, and potential data breaches.

AWS S3 is a leading tool in the Cloud Security space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing s3 bucket security audit and remediation properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure AWS S3

- Install AWS S3 using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure AWS S3 to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/aws-s3-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — AWS S3 installation

```
# Install AWS S3
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "AWS S3" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
aws s3 --version || aws s3 version

# Run initial health check
aws s3 --help

# Configure authentication if required
export AWS_S3_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# aws s3.yml or .aws s3.yml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: aws-s3-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yaml — .github/workflows/aws_s3.yml

```
name: S3 Bucket Security Audit and Remediation - AWS S3

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Cloud Security Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run AWS S3 Scan
        run: |
          # Install and configure AWS S3
          echo "Running AWS S3 security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: aws_s3-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## AWS S3 Security Scan Results\n See workflow artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
aws s3 scan \
```

```
--severity CRITICAL,HIGH \  
--format json \  
--output results.json \  
--exit-code 1 \  
.  
  
# Parse results programmatically  
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |  
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |  
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:  
  .FixedVersion}'  
  
# Generate HTML report  
aws s3 scan --format template \  
  --template @html.tpl --output report.html .  
  
# Fail pipeline only on CRITICAL  
aws s3 scan --exit-code 1 --severity CRITICAL .
```

Step 5: Remediation Workflow

When AWS S3 finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```
# Create GitHub issue for each critical finding  
cat results.json | jq -r '.Results[].Vulnerabilities[] |  
  select(.Severity == "CRITICAL") |  
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +  
  " | Fix: " + (.FixedVersion // "No fix available")' | \  
while read line; do  
  gh issue create --title "$line" --label "security,critical" --body "Auto-  
generated from AWS S3 scan"  
done  
  
# Track remediation metrics  
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length ]'  
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |  
  select(.Severity=="CRITICAL")] | length )'  
echo "Total: $TOTAL | Critical: $CRITICAL"
```

Ultra-Deep Explanation

AWS S3 Architecture and Detection Engine

AWS S3 uses sophisticated analysis techniques to detect security issues in cloud security contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Cloud Security Security Principles

Cloud Security security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). AWS S3 implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin AWS S3 versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating AWS S3's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 44 · CI/CD SECURITY**Secure Docker Build with BuildKit and SBOM**

★ ★ Intermediate · Tools: Docker BuildKit, Cosign, Syft, Trivy

Objective

Master Secure Docker Build with BuildKit and SBOM using Docker BuildKit, Cosign, Syft, Trivy to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

CI/CD Security is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper ci/cd security controls expose themselves to significant security risks, compliance violations, and potential data breaches.

Docker BuildKit is a leading tool in the CI/CD Security space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing secure docker build with buildkit and sbom properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure Docker BuildKit

- Install Docker BuildKit using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure Docker BuildKit to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/docker buildkit-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — Docker BuildKit installation

```
# Install Docker BuildKit
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "Docker BuildKit" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
docker buildkit --version || docker buildkit version

# Run initial health check
docker buildkit --help

# Configure authentication if required
export DOCKER_BUILDKIT_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# docker buildkit.yaml or .docker buildkit.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: docker buildkit-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yaml — .github/workflows/docker buildkit.yml

```
name: Secure Docker Build with BuildKit and SBOM - Docker BuildKit

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: CI/CD Security Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run Docker BuildKit Scan
        run: |
          # Install and configure Docker BuildKit
          echo "Running Docker BuildKit security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: docker buildkit-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## Docker BuildKit Security Scan Results\n See workflow
artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
docker buildkit scan \
```

```
--severity CRITICAL,HIGH \  
--format json \  
--output results.json \  
--exit-code 1 \  
.  
  
# Parse results programmatically  
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |  
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |  
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:  
  .FixedVersion}'  
  
# Generate HTML report  
docker buildkit scan --format template \  
  --template @html.tpl --output report.html .  
  
# Fail pipeline only on CRITICAL  
docker buildkit scan --exit-code 1 --severity CRITICAL .
```

Step 5: Remediation Workflow

When Docker BuildKit finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```
# Create GitHub issue for each critical finding  
cat results.json | jq -r '.Results[].Vulnerabilities[] |  
  select(.Severity == "CRITICAL") |  
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +  
  " | Fix: " + (.FixedVersion // "No fix available")' | \  
while read line; do  
  gh issue create --title "$line" --label "security,critical" --body "Auto-  
generated from Docker BuildKit scan"  
done  
  
# Track remediation metrics  
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length ]'  
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |  
  select(.Severity=="CRITICAL")] | length )'  
echo "Total: $TOTAL | Critical: $CRITICAL"
```

Ultra-Deep Explanation

Docker BuildKit Architecture and Detection Engine

Docker BuildKit uses sophisticated analysis techniques to detect security issues in ci/cd security contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: CI/CD Security Security Principles

CI/CD Security security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). Docker BuildKit implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin Docker BuildKit versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating Docker BuildKit's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 45 · PENETRATION TESTING**Kubernetes Penetration Testing with kube-hunter**

☆☆☆ Advanced · Tools: kube-hunter, Kubernetes, Kubectl

Objective

Master Kubernetes Penetration Testing with kube-hunter using kube-hunter, Kubernetes, Kubectl to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Penetration Testing is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper penetration testing controls expose themselves to significant security risks, compliance violations, and potential data breaches.

kube-hunter is a leading tool in the Penetration Testing space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing kubernetes penetration testing with kube-hunter properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure kube-hunter

- Install kube-hunter using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure kube-hunter to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/kube-hunter-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

☑ Complete Solution

Step 1: Installation and Initial Setup

bash — kube-hunter installation

```
# Install kube-hunter
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "kube-hunter" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
kube-hunter --version || kube-hunter version

# Run initial health check
kube-hunter --help

# Configure authentication if required
export KUBE_HUNTER_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# kube-hunter.yaml or .kube-hunter.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: kube-hunter-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yml — .github/workflows/kube-hunter.yml

```
name: Kubernetes Penetration Testing with kube-hunter - kube-hunter

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Penetration Testing Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run kube-hunter Scan
        run: |
          # Install and configure kube-hunter
          echo "Running kube-hunter security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: kube-hunter-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## kube-hunter Security Scan Results\n See workflow artifacts
for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
kube-hunter scan \
```

```

--severity CRITICAL,HIGH \
--format json \
--output results.json \
--exit-code 1 \
.

# Parse results programmatically
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:
  .FixedVersion}'

# Generate HTML report
kube-hunter scan --format template \
  --template @html.tpl --output report.html .

# Fail pipeline only on CRITICAL
kube-hunter scan --exit-code 1 --severity CRITICAL .

```

Step 5: Remediation Workflow

When kube-hunter finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```

# Create GitHub issue for each critical finding
cat results.json | jq -r '.Results[].Vulnerabilities[] |
  select(.Severity == "CRITICAL") |
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +
  " | Fix: " + (.FixedVersion // "No fix available")' | \
  while read line; do
    gh issue create --title "$line" --label "security,critical" --body "Auto-
generated from kube-hunter scan"
  done

# Track remediation metrics
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length')
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |
  select(.Severity=="CRITICAL")] | length')
echo "Total: $TOTAL | Critical: $CRITICAL"

```

Ultra-Deep Explanation

kube-hunter Architecture and Detection Engine

kube-hunter uses sophisticated analysis techniques to detect security issues in penetration testing contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false

positive rates. Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Penetration Testing Security Principles

Penetration Testing security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). kube-hunter implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin kube-hunter versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating kube-hunter's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 46 · MONITORING

SIEM Integration with Splunk for DevSecOps

★ ★ ★ Advanced · Tools: Splunk, Fluentd, AWS CloudTrail

Objective

Master SIEM Integration with Splunk for DevSecOps using Splunk, Fluentd, AWS CloudTrail to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Monitoring is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper monitoring controls expose themselves to significant security risks, compliance violations, and potential data breaches.

Splunk is a leading tool in the Monitoring space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing siem integration with splunk for devsecops properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure Splunk

- Install Splunk using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure Splunk to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/splunk-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

☑ Complete Solution

Step 1: Installation and Initial Setup

bash — Splunk installation

```
# Install Splunk
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "Splunk" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
splunk --version || splunk version

# Run initial health check
splunk --help

# Configure authentication if required
export SPLUNK_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# splunk.yaml or .splunk.yml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: splunk-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yml — .github/workflows/splunk.yml

```
name: SIEM Integration with Splunk for DevSecOps - Splunk

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Monitoring Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run Splunk Scan
        run: |
          # Install and configure Splunk
          echo "Running Splunk security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: splunk-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## Splunk Security Scan Results\n See workflow artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
splunk scan \
```

```
--severity CRITICAL,HIGH \  
--format json \  
--output results.json \  
--exit-code 1 \  
.  
  
# Parse results programmatically  
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |  
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |  
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:  
  .FixedVersion}'  
  
# Generate HTML report  
splunk scan --format template \  
  --template @html.tpl --output report.html .  
  
# Fail pipeline only on CRITICAL  
splunk scan --exit-code 1 --severity CRITICAL .
```

Step 5: Remediation Workflow

When Splunk finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```
# Create GitHub issue for each critical finding  
cat results.json | jq -r '.Results[].Vulnerabilities[] |  
  select(.Severity == "CRITICAL") |  
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +  
  " | Fix: " + (.FixedVersion // "No fix available")' | \  
while read line; do  
  gh issue create --title "$line" --label "security,critical" --body "Auto-  
generated from Splunk scan"  
done  
  
# Track remediation metrics  
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length ]'  
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |  
  select(.Severity=="CRITICAL")] | length )'  
echo "Total: $TOTAL | Critical: $CRITICAL"
```

Ultra-Deep Explanation

Splunk Architecture and Detection Engine

Splunk uses sophisticated analysis techniques to detect security issues in monitoring contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Monitoring Security Principles

Monitoring security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). Splunk implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin Splunk versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating Splunk's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 47 · POLICY AS CODE

Open Policy Agent for Microservice Authorization

★ ★ ★ Advanced · Tools: OPA, Rego, Envoy, Kubernetes

Objective

Master Open Policy Agent for Microservice Authorization using OPA, Rego, Envoy, Kubernetes to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Policy as Code is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper policy as code controls expose themselves to significant security risks, compliance violations, and potential data breaches.

OPA is a leading tool in the Policy as Code space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing open policy agent for microservice authorization properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure OPA

- Install OPA using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure OPA to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/opa-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — OPA installation

```
# Install OPA
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "OPA" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
opa --version || opa version

# Run initial health check
opa --help

# Configure authentication if required
export OPA_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# opa.yaml or .opa.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: opa-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yaml — .github/workflows/opa.yml

```
name: Open Policy Agent for Microservice Authorization - OPA

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Policy as Code Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run OPA Scan
        run: |
          # Install and configure OPA
          echo "Running OPA security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: opa-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## OPA Security Scan Results\n See workflow artifacts for
details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
opa scan \
```

```
--severity CRITICAL,HIGH \
--format json \
--output results.json \
--exit-code 1 \
.

# Parse results programmatically
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:
  .FixedVersion}'

# Generate HTML report
opa scan --format template \
  --template @html.tpl --output report.html .

# Fail pipeline only on CRITICAL
opa scan --exit-code 1 --severity CRITICAL .
```

Step 5: Remediation Workflow

When OPA finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```
# Create GitHub issue for each critical finding
cat results.json | jq -r '.Results[].Vulnerabilities[] |
  select(.Severity == "CRITICAL") |
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +
  " | Fix: " + (.FixedVersion // "No fix available")' | \
while read line; do
  gh issue create --title "$line" --label "security,critical" --body "Auto-
generated from OPA scan"
done

# Track remediation metrics
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length'] )
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |
select(.Severity=="CRITICAL")] | length')
echo "Total: $TOTAL | Critical: $CRITICAL"
```

Ultra-Deep Explanation

OPA Architecture and Detection Engine

OPA uses sophisticated analysis techniques to detect security issues in policy as code contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Policy as Code Security Principles

Policy as Code security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). OPA implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin OPA versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating OPA's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 48 · VULNERABILITY MANAGEMENT

End-to-End DevSecOps Pipeline Architecture

☆☆☆ Advanced · Tools: GitHub Actions, Trivy, SonarQube, ArgoCD

Objective

Master End-to-End DevSecOps Pipeline Architecture using GitHub Actions, Trivy, SonarQube, ArgoCD to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Vulnerability Management is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper vulnerability management controls expose themselves to significant security risks, compliance violations, and potential data breaches.

GitHub Actions is a leading tool in the Vulnerability Management space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing end-to-end devsecops pipeline architecture properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure GitHub Actions

- Install GitHub Actions using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure GitHub Actions to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/github actions-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — GitHub Actions installation

```
# Install GitHub Actions
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "GitHub Actions" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
github actions --version || github actions version

# Run initial health check
github actions --help

# Configure authentication if required
export GITHUB_ACTIONS_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# github actions.yaml or .github actions.yml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: github actions-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yaml — .github/workflows/github actions.yml

```
name: End-to-End DevSecOps Pipeline Architecture - GitHub Actions

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Vulnerability Management Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run GitHub Actions Scan
        run: |
          # Install and configure GitHub Actions
          echo "Running GitHub Actions security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: github actions-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## GitHub Actions Security Scan Results\n See workflow
artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
github actions scan \
```

```
--severity CRITICAL,HIGH \  
--format json \  
--output results.json \  
--exit-code 1 \  
.  
  
# Parse results programmatically  
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |  
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |  
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:  
  .FixedVersion}'  
  
# Generate HTML report  
github actions scan --format template \  
  --template @html.tpl --output report.html .  
  
# Fail pipeline only on CRITICAL  
github actions scan --exit-code 1 --severity CRITICAL .
```

Step 5: Remediation Workflow

When GitHub Actions finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```
# Create GitHub issue for each critical finding  
cat results.json | jq -r '.Results[].Vulnerabilities[] |  
  select(.Severity == "CRITICAL") |  
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +  
  " | Fix: " + (.FixedVersion // "No fix available")' | \  
while read line; do  
  gh issue create --title "$line" --label "security,critical" --body "Auto-  
generated from GitHub Actions scan"  
done  
  
# Track remediation metrics  
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length']  
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |  
  select(.Severity=="CRITICAL")] | length')  
echo "Total: $TOTAL | Critical: $CRITICAL"
```

Ultra-Deep Explanation

GitHub Actions Architecture and Detection Engine

GitHub Actions uses sophisticated analysis techniques to detect security issues in vulnerability management contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false

positive rates. Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Vulnerability Management Security Principles

Vulnerability Management security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). GitHub Actions implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin GitHub Actions versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating GitHub Actions's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 49 · CLOUD SECURITY

AWS EKS Security Hardening

★ ★ ★ Advanced · Tools: AWS EKS, IAM, IRSA, kube-bench

Objective

Master AWS EKS Security Hardening using AWS EKS, IAM, IRSA, kube-bench to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Cloud Security is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper cloud security controls expose themselves to significant security risks, compliance violations, and potential data breaches.

AWS EKS is a leading tool in the Cloud Security space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing aws eks security hardening properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure AWS EKS

- Install AWS EKS using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure AWS EKS to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create `.github/workflows/aws-eks-scan.yml`
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — AWS EKS installation

```
# Install AWS EKS
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "AWS EKS" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
aws eks --version || aws eks version

# Run initial health check
aws eks --help

# Configure authentication if required
export AWS_EKS_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# aws-eks.yml or .aws-eks.yml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: aws-eks-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yml — .github/workflows/aws_eks.yml

```
name: AWS EKS Security Hardening - AWS EKS

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Cloud Security Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run AWS EKS Scan
        run: |
          # Install and configure AWS EKS
          echo "Running AWS EKS security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: aws_eks-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## AWS EKS Security Scan Results\n See workflow artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
aws eks scan \
```

```
--severity CRITICAL,HIGH \  
--format json \  
--output results.json \  
--exit-code 1 \  
.  
  
# Parse results programmatically  
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |  
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |  
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:  
  .FixedVersion}'  
  
# Generate HTML report  
aws eks scan --format template \  
  --template @html.tpl --output report.html .  
  
# Fail pipeline only on CRITICAL  
aws eks scan --exit-code 1 --severity CRITICAL .
```

Step 5: Remediation Workflow

When AWS EKS finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```
# Create GitHub issue for each critical finding  
cat results.json | jq -r '.Results[].Vulnerabilities[] |  
  select(.Severity == "CRITICAL") |  
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +  
  " | Fix: " + (.FixedVersion // "No fix available")' | \  
while read line; do  
  gh issue create --title "$line" --label "security,critical" --body "Auto-  
generated from AWS EKS scan"  
done  
  
# Track remediation metrics  
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length ]'  
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |  
  select(.Severity=="CRITICAL")] | length )'  
echo "Total: $TOTAL | Critical: $CRITICAL"
```

Ultra-Deep Explanation

AWS EKS Architecture and Detection Engine

AWS EKS uses sophisticated analysis techniques to detect security issues in cloud security contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Cloud Security Security Principles

Cloud Security security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). AWS EKS implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin AWS EKS versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating AWS EKS's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

ASSIGNMENT 50 · CAPSTONE

Complete DevSecOps Pipeline: End-to-End Project

★★★★ Expert · Tools: All Tools: Gitleaks, Trivy, SonarQube, Vault, ArgoCD, Falco

Objective

Master Complete DevSecOps Pipeline: End-to-End Project using All Tools: Gitleaks, Trivy, SonarQube, Vault, ArgoCD, Falco to strengthen DevSecOps pipelines and enforce security controls in modern cloud-native environments. This assignment provides hands-on experience with real-world security scenarios encountered in production systems.

Background & Theory

Capstone is a critical discipline in modern DevSecOps practice. Organizations that fail to implement proper capstone controls expose themselves to significant security risks, compliance violations, and potential data breaches.

All Tools: Gitleaks is a leading tool in the Capstone space. Understanding its architecture, configuration, and integration patterns is essential for DevSecOps engineers building secure delivery pipelines.

This assignment maps to multiple industry frameworks including NIST SP 800-53, CIS Benchmarks, OWASP, and the DevSecOps Maturity Model (DSOMM). Each control implemented here directly addresses specific framework requirements, providing dual benefit: security improvement and compliance evidence.

Real-world incidents demonstrate the cost of not implementing these controls. By implementing complete devsecops pipeline: end-to-end project properly, teams can detect, prevent, and respond to security issues before they impact production systems and customers.

Tasks

Task 1: Install and Configure All Tools: Gitleaks

- Install All Tools: Gitleaks using the official package manager or container image
- Configure initial settings and authentication
- Verify installation by running a test scan or health check
- Review the output format and understand key fields

Task 2: Integrate with Application Pipeline

- Create a sample application with intentional security issues
- Configure All Tools: Gitleaks to scan the application in CI
- Analyze findings and prioritize remediation by severity
- Fix identified issues and verify clean scan results

Task 3: Production-Ready GitHub Actions Integration

- Create .github/workflows/all tools: gitleaks-scan.yml
- Configure pipeline gates to fail on critical findings
- Set up notifications for security team (Slack/email)
- Add SARIF upload to GitHub Security dashboard

Complete Solution

Step 1: Installation and Initial Setup

bash — All Tools: Gitleaks installation

```
# Install All Tools: Gitleaks
# Follow official documentation for latest version

# For container-based tools:
docker pull $(echo "All Tools: Gitleaks" | tr '[:upper:]' '[:lower:]'):latest

# Verify installation
all tools: gitleaks --version || all tools: gitleaks version

# Run initial health check
all tools: gitleaks --help

# Configure authentication if required
export ALL_TOOLS_GITLEAKS_TOKEN="your-token-here"
```

Step 2: Basic Scanning Configuration

yaml — Configuration file

```
# all tools: gitleaks.yaml or .all tools: gitleaks.yaml
version: "1.0"

scan:
  enabled: true
  severity: ["CRITICAL", "HIGH", "MEDIUM"]
  output:
    format: sarif
    file: all tools: gitleaks-results.sarif

filters:
  exclude:
    - tests/**
    - vendor/**
    - node_modules/**

notifications:
  on_failure: true
  channels:
    - slack
```

Step 3: GitHub Actions Workflow

yml — .github/workflows/all tools: gitleaks.yml

```
name: Complete DevSecOps Pipeline: End-to-End Project - All Tools: Gitleaks

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 8 * * 1' # Weekly Monday morning scan

jobs:
  security-scan:
    name: Capstone Scan
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run All Tools: Gitleaks Scan
        run: |
          # Install and configure All Tools: Gitleaks
          echo "Running All Tools: Gitleaks security scan..."
          # Add tool-specific commands here

      - name: Upload SARIF Results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: all tools: gitleaks-results.sarif

      - name: Comment PR with results
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '## All Tools: Gitleaks Security Scan Results\n See workflow
artifacts for details.'
            })
```

Step 4: Advanced Configuration and Tuning

bash — Advanced scanning options

```
# Scan with additional context
all tools: gitleaks scan \
```

```
--severity CRITICAL,HIGH \  
--format json \  
--output results.json \  
--exit-code 1 \  
.  
  
# Parse results programmatically  
cat results.json | jq '.Results[] | select(.Vulnerabilities != null) |  
  .Vulnerabilities[] | select(.Severity == "CRITICAL") |  
  {id: .VulnerabilityID, pkg: .PkgName, version: .InstalledVersion, fix:  
  .FixedVersion}'  
  
# Generate HTML report  
all tools: gitleaks scan --format template \  
  --template @html.tpl --output report.html .  
  
# Fail pipeline only on CRITICAL  
all tools: gitleaks scan --exit-code 1 --severity CRITICAL .
```

Step 5: Remediation Workflow

When All Tools: Gitleaks finds security issues, the remediation workflow should be: (1) Triage — determine if finding is a true positive or false positive. (2) Prioritize — CVSS score, exploitability, affected systems. (3) Assign — create Jira ticket or GitHub issue with finding details. (4) Fix — update dependency version, fix misconfiguration, or rotate secret. (5) Verify — re-run scan to confirm remediation. (6) Document — record the fix in the change management system for audit.

bash — Automated remediation helpers

```
# Create GitHub issue for each critical finding  
cat results.json | jq -r '.Results[].Vulnerabilities[] |  
  select(.Severity == "CRITICAL") |  
  "CVE: " + .VulnerabilityID + " | Package: " + .PkgName +  
  " | Fix: " + (.FixedVersion // "No fix available")' | \  
while read line; do  
  gh issue create --title "$line" --label "security,critical" --body "Auto-  
generated from All Tools: Gitleaks scan"  
done  
  
# Track remediation metrics  
TOTAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] | length ]'  
CRITICAL=$(cat results.json | jq ' [.Results[].Vulnerabilities[] |  
  select(.Severity=="CRITICAL")] | length )'  
echo "Total: $TOTAL | Critical: $CRITICAL"
```

Ultra-Deep Explanation

All Tools: Gitleaks Architecture and Detection Engine

All Tools: Gitleaks uses sophisticated analysis techniques to detect security issues in capstone contexts. The tool aggregates data from multiple security databases and applies both signature-based and heuristic detection to minimize false negatives while managing false positive rates.

Understanding the detection methodology helps practitioners tune configurations appropriately for their environments.

Deep Dive: Capstone Security Principles

Capstone security is founded on core principles: defense in depth (multiple security layers), least privilege (minimum permissions required), fail secure (default to deny on error), separation of duties (no single point of control), and auditability (all actions logged and reviewable). All Tools: Gitleaks implements these principles in its scanning and reporting capabilities.

Integration with DevSecOps Maturity Model

The DevSecOps Maturity Model (DSOMM) defines four levels of security practice maturity. Level 1: Basic scanning in CI. Level 2: Automated policy enforcement. Level 3: Runtime monitoring and response. Level 4: Threat modeling and continuous improvement. This assignment addresses Level 1-2 practices, forming the foundation for higher maturity levels.

Compliance Framework Mapping

Controls implemented in this assignment map to: NIST SP 800-53 (RA-5 Vulnerability Scanning, SI-3 Malicious Code Protection, CA-7 Continuous Monitoring), CIS Controls v8 (CIS 7: Continuous Vulnerability Management), PCI-DSS v4.0 (Requirement 6: Develop and Maintain Secure Systems), and ISO 27001 (A.12.6.1 Technical Vulnerability Management). Document these mappings for audit purposes.

Metrics and KPIs for Security Programs

Key metrics to track: Mean Time to Detect (MTTD) — how quickly are vulnerabilities found; Mean Time to Remediate (MTTR) — how quickly are vulnerabilities fixed; Vulnerability Density — number of critical issues per 1000 lines of code; Security Debt Ratio — open critical issues vs total issues; Pipeline Gate Pass Rate — percentage of builds passing security gates. Track trends over time using Grafana dashboards fed by scan output JSON.

★ Best Practices

- Pin All Tools: Gitleaks versions in CI to ensure reproducible scans
- Integrate findings with ticketing systems (Jira, GitHub Issues) for tracking
- Set SLA targets: Critical within 24h, High within 7 days, Medium within 30 days
- Generate weekly security metrics reports for engineering leadership
- Combine multiple scanning tools for comprehensive coverage (SAST + DAST + SCA + IaC)
- Implement 'security as code' — version control all configurations

⚠ Common Mistakes to Avoid

- Scanning only on merge to main — scan on every pull request for early feedback
- Not investigating false positives — over time they erode trust in the tool
- Siloing security findings from developers — make results visible in developer workflows
- Not updating All Tools: Gitleaks's vulnerability database before scanning
- Treating all findings with equal urgency — prioritize by exploitability and business impact

